

# FAST-LIVO2: Fast, Direct LiDAR–Inertial–Visual Odometry

Chunran Zheng<sup>ID</sup>, *Student Member, IEEE*, Wei Xu<sup>ID</sup>, Zuhao Zou<sup>ID</sup>, Tong Hua<sup>ID</sup>, Chongjian Yuan<sup>ID</sup>,  
Dongjiao He<sup>ID</sup>, *Member, IEEE*, Bingyang Zhou, *Member, IEEE*, Zheng Liu<sup>ID</sup>, *Member, IEEE*,  
Jiarong Lin<sup>ID</sup>, *Member, IEEE*, Fangcheng Zhu<sup>ID</sup>, *Student Member, IEEE*, Yunfan Ren<sup>ID</sup>, *Student Member, IEEE*,  
Rong Wang, Fanle Meng, and Fu Zhang<sup>ID</sup>, *Member, IEEE*

**Abstract**—This paper presents FAST-LIVO2, a fast and direct LiDAR-inertial-visual odometry framework designed for accurate and robust state estimation in SLAM tasks, enabling real-time robotic applications. FAST-LIVO2 integrates IMU, LiDAR, and image data through an efficient error-state iterated Kalman filter (ESIKF). To address the dimensional mismatch between LiDAR and image measurements, we adopt a sequential update strategy. Efficiency is further enhanced using direct methods for LiDAR and visual data fusion: the LiDAR module registers raw points without extracting features, while the visual module minimizes photometric errors without relying on feature extraction. Both LiDAR and visual measurements are fused into a unified voxel map. The LiDAR module constructs the geometric structure, while the visual module links image patches to LiDAR points, enabling precise image alignment. Plane priors from LiDAR points improve alignment accuracy and are refined dynamically during the process. Additionally, an on-demand raycast operation and real-time image exposure estimation enhance robustness. Extensive experiments on benchmark and custom datasets demonstrate that FAST-LIVO2 outperforms state-of-the-art systems in accuracy, robustness, and efficiency. Key modules are validated, and we showcase three applications: UAV navigation highlighting real-time capabilities, airborne mapping demonstrating high accuracy, and 3D model rendering (mesh-based and NeRF-based) showcasing suitability for dense mapping. Code and datasets are open-sourced on GitHub to benefit the robotics community.

**Index Terms**—3-D reconstruction, aerial navigation, sensor fusion, simultaneous localization and mapping (SLAM).

## I. INTRODUCTION

IN RECENT years, simultaneous localization and mapping (SLAM) technology<sup>1</sup> has seen significant advancements, particularly in real-time 3-D reconstruction and localization in

unknown environments. Due to its ability to estimate poses and reconstruct maps in real time, SLAM has become indispensable for various robot navigation tasks. The localization process delivers crucial state feedback for the robot's onboard controllers, while the dense 3-D map provides key environmental information, such as free spaces and obstacles, essential for effective trajectory planning. A colored map also carries substantial semantic information, enabling a vivid representation of the real world that opens up vast potential applications, such as virtual and augmented reality, 3-D modeling, and robot–human interactions.

Currently, several SLAM frameworks have been successfully implemented with single-measurement sensors, primarily cameras [1], [2], [3], [4], or LiDAR [5], [6], [7]. Although visual and LiDAR SLAM have shown promise in their own domains, each has inherent limitations that constrain their performance in various scenarios.

Visual SLAM, leveraging cost-effective CMOS sensors and lenses, is capable of establishing accurate data associations, thereby achieving a certain level of localization accuracy. The abundance of color information further enriches the semantic perception. Further leveraging this enhanced scene comprehension; deep learning methods are employed for robust feature extraction and dynamic object filtering. However, the lack of direct depth measurement in visual SLAM necessitates concurrent optimization of map points via operations, such as triangulation or depth filtering, which introduces significant computational overhead that often limits map accuracy and density. Visual SLAM also encounters numerous other limitations, such as varying measurement noise across different scales, sensitivity to illumination changes, and the impact of texture-less environments on data association.

LiDAR SLAM, utilizing LiDAR sensors, obtains precise depth measurements directly, offering superior precision and efficiency in localization and mapping tasks compared with visual SLAM. Despite these strengths, LiDAR SLAM exhibits several significant shortcomings. On one hand, the point cloud maps it reconstructs, albeit detailed, lack color information, thereby reducing their information scale. On the other hand, LiDAR SLAM performance tends to deteriorate in environments presenting insufficient geometric constraints, such as narrow tunnels and a single and extended wall.

As the demand to operate intelligent robots in the real world grows, especially in environments that often lack structure or texture, it is becoming clear that existing systems relying on a single sensor cannot provide the accurate and robust pose estimation as required. To address this issue, the fusion of

Received 24 April 2024; revised 23 August 2024; accepted 1 October 2024. Date of publication 19 November 2024; date of current version 12 December 2024. This work was supported in part by the CETC ISA and in part by the Hong Kong General Research Fund (GRF) with under Grant 17206421. This article was recommended for publication by Associate Editor A. Nuechter and Editor J. Civera upon evaluation of the reviewers' comments. (*Corresponding author: Fu Zhang.*)

Chunran Zheng, Wei Xu, Zuhao Zou, Tong Hua, Chongjian Yuan, Dongjiao He, Bingyang Zhou, Zheng Liu, Jiarong Lin, Fangcheng Zhu, Yunfan Ren, and Fu Zhang are with the Mechatronics and Robotic Systems (MaRS) Laboratory, Department of Mechanical Engineering, University of Hong Kong, Hong Kong, SAR 999077, China (e-mail: fuzhang@hku.hk).

Rong Wang and Fanle Meng are with the Information Science Academy, China Electronics Technology Group Corporation, Beijing 100846, China.

Digital Object Identifier 10.1109/TRO.2024.3502198

<sup>1</sup><https://github.com/hku-mars/FAST-LIVO2>

commonly used sensors, such as LiDAR, camera, and IMU, is gaining increasing attention. This strategy not only combines the strengths of these sensors to provide enhanced pose estimation, but also aids in the construction of accurate, dense, and colored point cloud maps, even in environments where the performance of individual sensors degenerate.

Efficient and accurate LiDAR-inertial-visual odometry (LIVO) and mapping are still challenging problems.

- 1) The entire LIVO system is tasked with processing LiDAR measurements, consisting of hundreds to thousands of points per second, as well as high-rate, high-resolution images. The challenge of fully utilizing such a vast amount of data, particularly with limited onboard resources, necessitates exceptional computational efficiency.
- 2) Many existing systems typically incorporate a LiDAR-inertial odometry (LIO) subsystem and a visual-inertial odometry (VIO) subsystem, each necessitating the extraction of features from visual and LiDAR data respectively to reduce computational load. In environments that lack structure or texture, this extraction process often results in limited feature points. Furthermore, to optimize feature extraction, extensive engineering adaptations are essential to accommodate the variability in LiDAR scanning patterns and point densities.
- 3) To reduce computational demands and achieve tighter integration between camera and LiDAR measurements, a unified map is essential to manage sparse points and the observed high-resolution image measurements simultaneously. However, designing and maintaining such maps are particularly challenging considering the heterogeneous measurements of LiDAR and cameras.
- 4) To ensure the accuracy of the reconstructed colored point cloud, pose estimation needs to achieve pixel-level accuracy. Meeting this standard presents considerable challenges: proper hardware synchronization, rigorous pre-calibration of extrinsic parameters between LiDAR and cameras, precise recovery of exposure time, and a fusion strategy capable of reaching pixel-level accuracy in real time.

Motivated by these issues, we propose FAST-LIVO2, a high-efficiency LIVO system that tightly integrates LiDAR, image and IMU measurements through a sequentially updated error-state iterated Kalman filter (ESIKF). With the prior from IMU propagation, the system state is updated sequentially, first by the LiDAR measurements and then by the image measurements, both utilizing direct methods based on a single unified voxel map. Specifically, in the LiDAR update, the system registers raw points to the map to construct and update its geometric structure, and in the visual update, the system reuses LiDAR map points as the visual map points directly without extracting, triangulating, or optimizing any visual features from images. The chosen visual map points in the map are attached with reference image patches previously observed and then projected to the current image to align its pose by minimizing the direct photometric errors (i.e., sparse image alignment). To improve the accuracy in the image alignment, FAST-LIVO2 dynamically updates the reference patches and uses the plane priors obtained from LiDAR points. For improved computation efficiency, FAST-LIVO2 uses LiDAR points to identify visual map points visible from the current image and conduct an on-demanding voxel raycast in case of no LiDAR points. FAST-LIVO2 also estimates the exposure time in real time to handle illumination variation.

FAST-LIVO2 is developed based on FAST-LIVO first proposed in our previous work [8]. The new contributions compared to FAST-LIVO are listed as follows.

- 1) We propose an efficient ESIKF framework with sequential update to address the dimension mismatch between LiDAR and visual measurements, improving the robustness of FAST-LIVO that uses asynchronous updates.
- 2) We use (and even refine) plane priors from LiDAR points for improved accuracy. In contrast, FAST-LIVO assumes all pixels in a patch share the same depth, a wild assumption significantly reducing the accuracy of affine warping in image alignment.
- 3) We propose a reference patch update strategy to improve the accuracy of image alignment, by selecting high-quality, inlier reference patches that have large parallax and sufficient texture details. FAST-LIVO selects the reference patch based on proximity to the current view, often resulting in low-quality reference patches degrading the accuracy.
- 4) We conduct online exposure time estimation for handling environment illumination variation. FAST-LIVO did not address this issue, leading to poor convergence in image alignment under significant lighting changes.
- 5) We propose on-demand voxel raycasting to enhance the system robustness in the absence of LiDAR point measurements caused by LiDAR close proximity blind zones, an issue not considered in FAST-LIVO.

Each of the above-mentioned contributions are evaluated in comprehensive ablation studies to verify their effectiveness. We implement the proposed system as practical open software, meticulously optimized for real-time operation on both Intel and ARM processors. The system is versatile, supporting multiline spinning LiDARs, emerging solid-state LiDARs with unconventional scanning patterns, as well as both pinhole cameras and various fisheye cameras.

Besides, we conduct extensive experiments on 25 sequences of public datasets (i.e., Hilti and NTU-VIRAL datasets), alongside various representative private datasets, enabling a comparison with other state-of-the-art (SOTA) SLAM systems (e.g., R3LIVE, LVI-SAM, FAST-LIO2, etc). Both qualitative and quantitative results demonstrate that our proposed system significantly outpaces other counterparts in terms of accuracy and robustness at a reduced computation cost.

Taking a step further to underline the real-world applicability and versatility of our system, we deploy three distinctive applications. First, fully onboard autonomous UAV navigation, demonstrating the system's real-time capabilities, marks a pioneering instance of employing a LiDAR-inertial-visual system for real-world autonomous UAV flights. Second, airborne mapping showcases the system's pixel-level precision under structure-less environments in practical use. Finally, the high-quality generation of mesh, texturing, and NeRF models underscores the system's suitability for rendering tasks. We make our code and dataset available on GitHub.

## II. RELATED WORKS

### A. Direct Methods

Direct methods stand out as a prominent approach for fast pose estimation in both visual and LiDAR SLAM. Unlike feature-based methods [5], [6], [9], [10] which necessitate the

extraction of salient feature points (e.g., corners and edge pixels in images; plane and edge points in LiDAR scans) and the generation of robust descriptors for matching, direct methods directly leverage raw measurements to optimize the sensor pose [11] by minimizing an error function based on photometric error or point-to-plane residuals, e.g., [3], [12], [13], [14]. By eliminating the time-consuming feature extraction and matching, direct methods offer fast pose estimation. Nonetheless, the absence of feature matching requires fairly accurate state prior estimation to avoid local minima.

Direct methods in visual SLAM can be broadly categorized into dense direct, semidense direct, and sparse direct methods. Dense direct methods, predominantly adopted for RGB-D cameras with full depth measurements as exemplified by authors in [15], [16], and [17], apply image-to-model alignment for pose estimation. In contrast, semidense direct methods [3], [18] implement direct image alignment by capitalizing on pixels with significant gray-level gradients for estimation. Sparse direct methods [2], [12] focus on delivering accurate state estimation through only a few well-selected raw patches, thus further diminishing the computational burden in comparison to both dense and semidense direct methods.

Unlike direct visual SLAM methods, direct LiDAR SLAM systems [13], [14], [19], [20] do not distinguish between dense and sparse approaches and commonly use spatially downsampled or temporally downsampled raw points in each scan to construct constraints for pose optimization.

In our work, we harness the principles of the direct method for both LiDAR and visual modules. The LiDAR module of our system is adapted from VoxelMap [14], and the visual module is based on a variant of sparse direct method [12]. While drawing inspiration from sparse direct image alignment in [12], our visual module differs by reutilizing the LiDAR points as visual map points, thus mitigating the intensive backend computations (i.e., feature alignment, sliding window optimization and/or depth filtering).

## B. LiDAR–Visual(–Inertial) SLAM

The incorporation of multiple sensors in LiDAR–visual–inertial SLAM equips the system with the capability to handle a wide range of challenging environments, particularly when one sensor experiences failure or partial degeneration. Motivated by this, the research community has seen the emergence of various LiDAR–visual–inertial SLAM systems. Existing methods can generally be divided into two categories: loosely coupled and tightly coupled. The classification can be determined from two perspectives: the state estimation level and the raw measurement level. At the state estimation level, the key is whether the estimate from one sensor serves as an optimization objective in another sensor’s model. At the raw measurement level, it involves whether raw data from different sensors are combined.

Zhang et al. [21] proposed a LiDAR–visual–inertial SLAM system that is loosely coupled at the state estimation level. In this system, VIO subsystem only provides the initial pose for the scan registration in LIO subsystem, instead of being optimized jointly with the scan registration. VIL-SLAM [22] employs a similar loosely coupled method, not utilizing joint optimization of LiDAR, camera, and IMU measurements.

Some systems (e.g., DEMO [23], LIMO [24], CamVox [25], [26]) use 3-D LiDAR points to provide depth measurements for the visual module [1], [4], [27]. While these systems exhibit

measurement-level tight coupling, they remain loosely coupled in state estimation, primarily due to the absence of constraints directly derived from LiDAR measurements at state estimation. Another issue arises as 3-D LiDAR points do not have a one-to-one correspondence with 2-D image feature points and/or lines due to mismatched resolutions. This mismatch requires interpolation in depth association, introducing potential errors. To address this, DVL-SLAM [28] employs a direct method for visual tracking, wherein the LiDAR points are directly projected into the image to ascertain the depth of corresponding pixel positions.

The works mentioned previously have not achieved tight coupling at the state estimation level. In pursuit of higher accuracy and robustness, many recent studies have emerged that jointly optimize sensor data in a tightly coupled manner. To name a few, LIC-Fusion [29] tightly fuses IMU measurements, sparse visual features, and LiDAR plane and edge features based on the MSCKF [30] framework. The subsequent LIC-Fusion2.0 [31] enhances LiDAR pose estimation by implementing plane-feature tracking within a sliding window. VILENS [32] offers a joint optimization of visual, LiDAR, and inertial data through a unified factor graph, relying on fixed lag smoothing. R2LIVE [33] tightly fuses the LiDAR, camera, and IMU measurements in an on-manifold iterated Kalman filter [34]. For the VIO subsystem in R2LIVE, a sliding window optimization is used to triangulate the locations of visual features in the map.

Several systems achieve complete tight coupling at both the measurement and state estimation levels. LVI-SAM [35] fuses the LiDAR, visual, and inertial sensors in a tightly coupled smoothing and mapping framework, which is built atop a factor graph. The VIO subsystem performs visual feature tracking and extracts feature depth using LiDAR scans. R3LIVE [36] constructs the geometric structure of the global map by LIO and renders map texture by VIO. These two subsystems estimate the system state jointly by fusing their respective LiDAR or visual data with IMUs. The advanced version, R3LIVE++ [37], estimates exposure time in real time and conducts photometric calibration in advance [38], which enables the system to recover the radiance of map points. Unlike most previously mentioned LiDAR-inertial-visual systems that rely on feature-based methods for both LIO and VIO subsystems, R3LIVE series [36], [37] adopt direct methods for both without feature extraction, enabling them to capture subtle environmental features even in texture-less or structure-less scenarios.

Our system also jointly estimates the state using LiDAR, image and IMU data, and maintains a tightly coupled voxel map at the measurement level. Furthermore, our system uses direct methods, harnessing raw LiDAR points for LiDAR scan registration and employing raw image patches for visual tracking. The key difference between our system and R3LIVE (or R3LIVE++) is that R3LIVE (and R3LIVE++) operate at an individual pixel level in the VIO, while our system operates at image patch levels. This difference bestows our system with marked advantages. First, in terms of robustness, our methodology uses a simplified, one-step frame-to-map sparse image alignment for pose estimation, mitigating the heavy reliance on an accurate initial state that has to be obtained by a frame-to-frame optical flow in R3LIVES. Consequently, our system simplifies and improves upon the two-stage frame-to-frame and frame-to-map operations in R3LIVE. Second, from a computational standpoint, the VIO in R3LIVE predominantly employs a dense direct method which

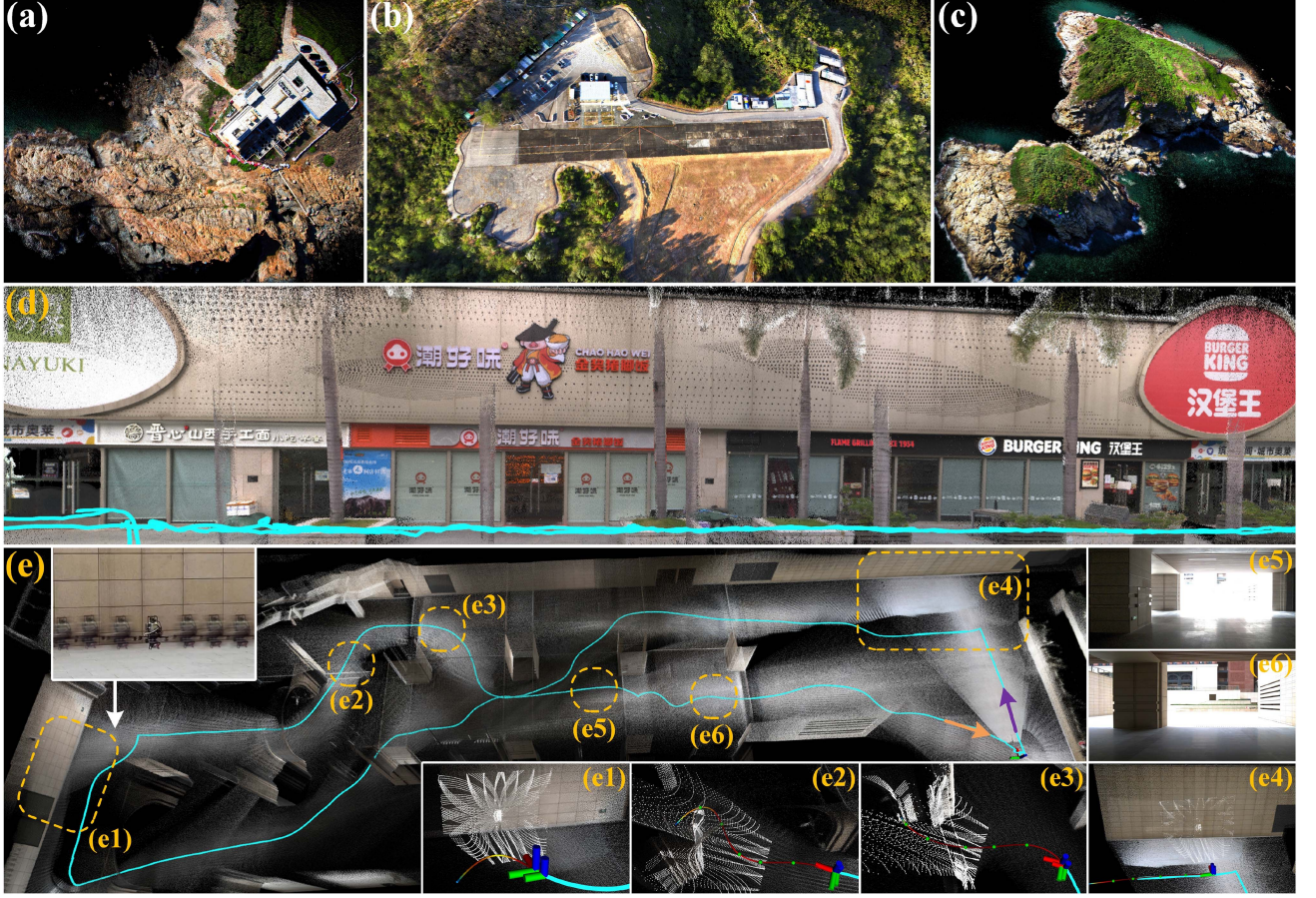


Fig. 1. FAST-LIVO2 mapping results generated in real time. (a)–(c) showcase airborne mapping, (d) represents a retail street collected with a handheld device, and (e) demonstrates an experiment where a UAV carrying a LiDAR, camera, and inertial sensor perform real-time state estimation (i.e., FAST-LIVO2), trajectory planning, and tracking control all on its onboard computer. In (d)–(e), blue lines represent the computed trajectory. In (e1)–(e4), white points indicate the LiDAR scan at that moment, and colored lines depict the planned trajectory. (e1) and (e4) mark areas of LiDAR degeneration. (e2) and (e3) show obstacle avoidance. (e5) and (e6) depict the camera first-person view from indoor to outdoor, highlighting large illumination variation from sudden overexposure to normal (see our accompanying video on YouTube).<sup>2</sup>

is computationally expensive, necessitating extensive points for residual construction and rendering. In contrast, our sparse direct method provides enhanced computational efficiency. Finally, our system exploits information at the resolution of raw image patches, whereas R3LIVE is capped at the resolution of its point map.

The visual module of our system is most similar to DV-LOAM [39], SDV-LOAM [40], and LVIO-Fusion [41], which projects LiDAR points attached with patches into a new image and tracks the image by minimizing the direct photometric error. However, they have several key differences, such as the use of separate maps for vision and LiDAR, reliance on the assumption of constant depth in patch warping in the visual module, loosely coupling at the state estimation level, and two stages of frame-to-frame and frame-to-keyframes for image alignment. In contrast, our system tightly integrates frame-to-map image alignment, LiDAR scan registration, and IMU measurements in an iterated Kalman filter. Besides, thanks to the single unified map for both LiDAR and visual modules, our system can directly employ the plane priors provided by the LiDAR points to accelerate the image alignment.

<sup>2</sup><https://youtu.be/6dF2DzgbtLY>

### III. SYSTEM OVERVIEW

The overview of our system is shown in Fig. 2, which contains four sections: ESIKF (see Section IV), local mapping (see Section V), LiDAR measurement model (see Section VI), and visual measurement model (see Section VII).

The asynchronously sampled LiDAR points are first recombined into scans at the camera's sampling time through scan recombination. Then, we tightly couple the LiDAR, image and inertial measurements via an ESIKF with sequential state update, where the system state is updated sequentially, first by the LiDAR measurements and then by the image measurements, both utilizing direct methods based on a single unified voxel map (see Section IV). To construct the LiDAR measurement model in the ESIKF update (see Section VI), we compute the frame-to-map point-to-plane residual. To establish visual measurement model (see Section VII), we extract the visual map points within the current FoV from the map, making use of visible voxel query and on-demand raycasting; after the extraction, we identify and discard outlier visual map points (e.g., points that are occluded or exhibit depth discontinuity); we then compute frame-to-map image photometric errors for visual update.

The local map for both visual and LiDAR updates is a voxel-map structure (see Section V): the LiDAR points construct and

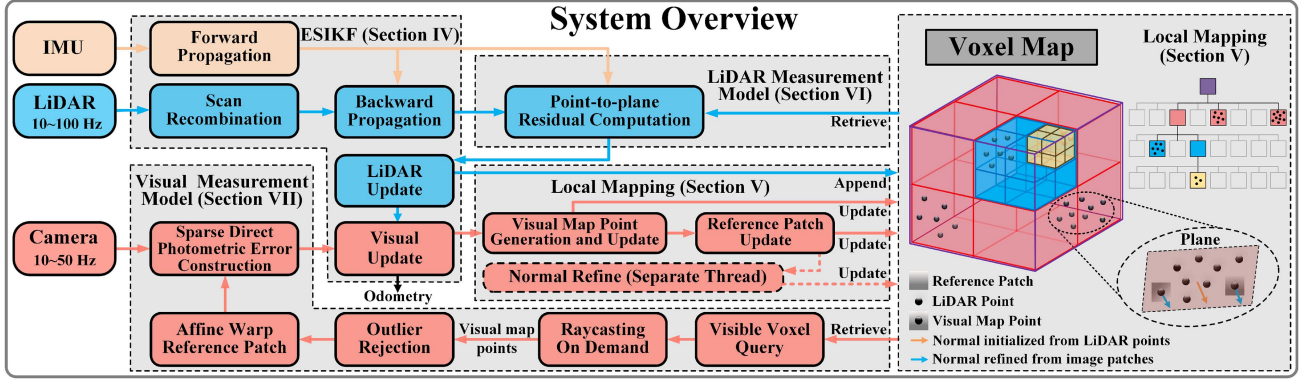


Fig. 2. System overview of FAST-LIVO2.

TABLE I  
SOME IMPORTANT NOTATIONS

| Notations                                        | Explanation                                                                |
|--------------------------------------------------|----------------------------------------------------------------------------|
| $\boxplus / \boxminus$                           | The encapsulated “boxplus” and “boxminus” operations on the state manifold |
| $^G(\cdot)$                                      | A vector $(\cdot)$ in global world frame                                   |
| $^C(\cdot)$                                      | A vector $(\cdot)$ in camera frame                                         |
| $^I\mathbf{T}_L$                                 | The extrinsic of LiDAR frame w.r.t. IMU frame                              |
| $^C\mathbf{T}_I$                                 | The extrinsic of IMU frame w.r.t. camera frame                             |
| $^G\mathbf{T}_I$                                 | The pose of IMU frame at time $k$ w.r.t. the global frame                  |
| $\mathbf{x}, \hat{\mathbf{x}}, \bar{\mathbf{x}}$ | The ground-truth, predicted and updated estimation of $\mathbf{x}$         |
| $\hat{\mathbf{x}}^\kappa$                        | The $\kappa$ -th update of $\mathbf{x}$                                    |
| $\delta\mathbf{x}$                               | The error state between ground-truth $\mathbf{x}$ and its estimation       |

update the map’s geometric structure, while the visual images append image patches to selected map points (i.e., visual map points) and update reference patches dynamically. The updated reference patches have their normal vectors further refined in a separate thread.

#### IV. ESIKF WITH SEQUENTIAL STATE UPDATE

This section outlines the system’s architecture, based on the sequentially updated ESIKF framework.

##### A. Notations and State Transition Model

In our system, we assume the time offsets among the three sensors (LiDAR, IMU, and camera) are known, which can be calibrated or synchronized in advance. We take IMU frame (denoted as  $I$ ) as the body frame and the first body frame as the global frame (denoted as  $G$ ). Besides, we assume that the three sensors are rigidly attached and the extrinsic, defined in Table I, are precalibrated. Then, the discrete state transition model at the  $i$ th IMU measurement is

$$\mathbf{x}_{i+1} = \mathbf{x}_i \boxplus (\Delta t \mathbf{f}(\mathbf{x}_i, \mathbf{u}_i, \mathbf{w}_i)) \quad (1)$$

where  $\Delta t$  is the IMU sample period, the state  $\mathbf{x}$ , input  $\mathbf{u}$ , process noise  $\mathbf{w}$ , and function  $\mathbf{f}$  are defined as follows:

$$\mathcal{M} \triangleq SO(3) \times \mathbb{R}^{16}, \dim(\mathcal{M}) = 19$$

$$\mathbf{x} \triangleq [{}^G\mathbf{R}_I^T \quad {}^G\mathbf{p}_I^T \quad {}^G\mathbf{v}_I^T \quad \mathbf{b}_g^T \quad \mathbf{b}_a^T \quad {}^G\mathbf{g}^T \quad \tau]^T \in \mathcal{M}$$

$$\mathbf{u} \triangleq [\omega_m^T \quad \mathbf{a}_m^T]^T, \quad \mathbf{w} \triangleq [\mathbf{n}_g^T \quad \mathbf{n}_a^T \quad \mathbf{n}_{b_g}^T \quad \mathbf{n}_{b_a}^T \quad n_\tau]^T$$

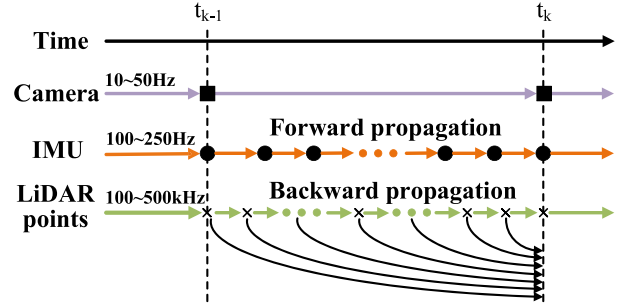


Fig. 3. Illustration of scan recombination, forward propagation and backward propagation applied to input data.

$$\mathbf{f}(\mathbf{x}, \mathbf{u}, \mathbf{w}) = \begin{bmatrix} \omega_m - \mathbf{b}_g - \mathbf{n}_g \\ {}^G\mathbf{v}_I + \frac{1}{2}({}^G\mathbf{R}_I(\mathbf{a}_m - \mathbf{b}_a - \mathbf{n}_a) + {}^G\mathbf{g})\Delta t \\ {}^G\mathbf{R}_I(\mathbf{a}_m - \mathbf{b}_a - \mathbf{n}_a) + {}^G\mathbf{g} \\ \mathbf{n}_{b_g} \\ \mathbf{n}_{b_a} \\ \mathbf{0}_{3 \times 1} \\ n_\tau \end{bmatrix} \quad (2)$$

where  ${}^G\mathbf{R}_I$ ,  ${}^G\mathbf{p}_I$ , and  ${}^G\mathbf{v}_I$  denote the IMU attitude, position, and velocity in the global frame, respectively,  ${}^G\mathbf{g}$  is the gravity vector in the global frame,  $\tau$  is the inverse camera exposure time relative to the first frame,  $n_\tau$  is the Gaussian noise that models  $\tau$  as a random walk,  $\omega_m$  and  $\mathbf{a}_m$  are the raw IMU measurements,  $\mathbf{n}_g$  and  $\mathbf{n}_a$  are measurement noises in  $\omega_m$  and  $\mathbf{a}_m$ ,  $\mathbf{b}_a$  and  $\mathbf{b}_g$  are IMU bias, which are modeled as random walk driven by Gaussian noise  $\mathbf{n}_{b_g}$  and  $\mathbf{n}_{b_a}$ , respectively.

##### B. Scan Recombination

We employ the scan recombination to segment the high-frequency, sequentially sampled LiDAR raw points into distinct LiDAR scans at the camera sampling moments, as depicted in Fig. 3. This ensures both camera and LiDAR data are synchronized at the same frequency (e.g., 10 Hz), allowing for update of state at the same time.

---

**Algorithm 1:** Sequential State Update.
 

---

```

1 Scan recombination to synchronize LiDAR and image
  data at the camera rate;
2 Forward propagation to obtain state prediction  $\hat{\mathbf{x}}$  and
  its covariance  $\hat{\mathbf{P}}$ ;
3 Backward propagation for LiDAR points motion
  compensation;
4 // Point-to-plane LiDAR update
5  $\kappa = -1$ ,  $\hat{\mathbf{x}}^{\kappa=0} = \hat{\mathbf{x}}$ ;
6 repeat
7    $\kappa = \kappa + 1$ ;
8   Compute residual  $\mathbf{z}_l^\kappa$  and Jacobian  $\mathbf{H}_l^\kappa$ ;
9   Compute the state update  $\hat{\mathbf{x}}^{\kappa+1}$ ;
10 until  $\|\hat{\mathbf{x}}^{\kappa+1} \ominus \hat{\mathbf{x}}^\kappa\| < \epsilon$ ;
11  $\hat{\mathbf{x}} = \hat{\mathbf{x}}^{\kappa+1}$ ,  $\hat{\mathbf{P}} = (\mathbf{I} - \mathbf{K}\mathbf{H}) \hat{\mathbf{P}}$ ;
12 // Sparse direct visual update
13 level = -1;
14 repeat
15    $\kappa = -1$ ,  $\hat{\mathbf{x}}^{\kappa=0} = \hat{\mathbf{x}}$ ;
16   level = level + 1;
17   repeat
18      $\kappa = \kappa + 1$ ;
19     Compute residual  $\mathbf{z}_c^\kappa$  and Jacobian  $\mathbf{H}_c^\kappa$ ;
20     Compute the state update  $\hat{\mathbf{x}}^{\kappa+1}$ ;
21   until  $\|\hat{\mathbf{x}}^{\kappa+1} \ominus \hat{\mathbf{x}}^\kappa\| < \epsilon$ ;
22    $\hat{\mathbf{x}} = \hat{\mathbf{x}}^{\kappa+1}$ ;
23 until level >= 2;
24  $\bar{\mathbf{x}} = \hat{\mathbf{x}}^{\kappa+1}$ ,  $\bar{\mathbf{P}} = (\mathbf{I} - \mathbf{K}\mathbf{H}) \hat{\mathbf{P}}$ .
```

---

### C. Propagation

In the ESIKF framework, the state and covariance are propagated from time  $t_{k-1}$ , when the last LiDAR scan and image frame are received, to time  $t_k$ , when the current LiDAR scan and image frame are received. This forward propagation predicts the state at each IMU input  $\mathbf{u}_i$  during  $t_{k-1}$  and  $t_k$ , by setting the process noise  $\mathbf{w}_i$  in (1) to zero. Denote the propagated state as  $\hat{\mathbf{x}}$  and covariance as  $\hat{\mathbf{P}}$ , which will serve as a prior distribution for the subsequent update in Section IV-D. Moreover, to compensate for motion distortion, we conduct a backward propagation as in [42], ensuring points in a LiDAR scan are “measured” at the scan-end time  $t_k$ . Note that for notation simplification, we omit the subscript  $k$  in all state vectors.

### D. Sequential Update

The IMU propagated state  $\hat{\mathbf{x}}$  and covariance  $\hat{\mathbf{P}}$  impose a prior distribution for  $\mathbf{x}$ , the system state at time  $t_k$ , as follows:

$$\mathbf{x} \ominus \hat{\mathbf{x}} \sim \mathcal{N}(\mathbf{0}, \hat{\mathbf{P}}). \quad (3)$$

We denote the above-mentioned prior distribution as  $p(\mathbf{x})$  and the measurement models for the LiDAR and camera as

$$\begin{bmatrix} \mathbf{y}_l \\ \mathbf{y}_c \end{bmatrix} = \begin{bmatrix} \mathbf{h}_l(\mathbf{x}, \mathbf{v}_l) \\ \mathbf{h}_c(\mathbf{x}, \mathbf{v}_c) \end{bmatrix} \quad (4)$$

where  $\mathbf{v}_l \sim \mathcal{N}(\mathbf{0}, \Sigma_{\mathbf{v}_l})$  and  $\mathbf{v}_c \sim \mathcal{N}(\mathbf{0}, \Sigma_{\mathbf{v}_c})$  denote the measurement noises for the LiDAR and camera, respectively.

A standard ESIKF [43] would update the state  $\mathbf{x}$  using all the current measurements, including both LiDAR measurement

$\mathbf{y}_l$  and image measurements  $\mathbf{y}_c$ . However, LiDAR and image measurements are two different sensing modalities, whose data dimensions do not match. Furthermore, the fusion of image measurement may be performed at various levels of the image pyramid. To address the dimension mismatch and give more flexibility for each module, we propose a sequential update strategy. This strategy is theoretically equivalent to the standard update using all measurements, assuming statistical independence of LiDAR measurements  $\mathbf{y}_l$  and image measurements  $\mathbf{y}_c$  given the state vector  $\mathbf{x}$  (i.e., measurements corrupted by statistically independent noise).

To introduce the sequential update, we rewrite the total conditional distribution for the current state  $\mathbf{x}$  as

$$\begin{aligned} p(\mathbf{x} | \mathbf{y}_l, \mathbf{y}_c) &\propto p(\mathbf{x}, \mathbf{y}_l, \mathbf{y}_c) = p(\mathbf{y}_c | \mathbf{x}, \mathbf{y}_l) p(\mathbf{x}, \mathbf{y}_l) \\ &= p(\mathbf{y}_c | \mathbf{x}) \underbrace{p(\mathbf{y}_l | \mathbf{x}) p(\mathbf{x})}_{\propto p(\mathbf{x} | \mathbf{y}_l)}. \end{aligned} \quad (5)$$

Equation (5) implies that the total conditional distribution  $p(\mathbf{x} | \mathbf{y}_l, \mathbf{y}_c)$  can be obtained by two sequential Bayesian updates. The first step fuses only the LiDAR measurement  $\mathbf{y}_l$  with the IMU-propagated prior distribution  $p(\mathbf{x})$  to obtain the distribution  $p(\mathbf{x} | \mathbf{y}_l)$

$$p(\mathbf{x} | \mathbf{y}_l) \propto p(\mathbf{y}_l | \mathbf{x}) p(\mathbf{x}). \quad (6)$$

The second step then fuses the camera measurement  $\mathbf{y}_c$  with  $p(\mathbf{x} | \mathbf{y}_l)$  to obtain the final posterior distribution of  $\mathbf{x}$

$$p(\mathbf{x} | \mathbf{y}_l, \mathbf{y}_c) \propto p(\mathbf{y}_c | \mathbf{x}) p(\mathbf{x} | \mathbf{y}_l). \quad (7)$$

Interestingly, the two fusion in (6) and (7) follow the same form

$$q(\mathbf{x} | \mathbf{y}) \propto q(\mathbf{y} | \mathbf{x}) q(\mathbf{x}). \quad (8)$$

To conduct the fusion in (8) for either LiDAR or image measurements, we detail the prior distribution  $q(\mathbf{x})$  and measurement model  $q(\mathbf{y} | \mathbf{x})$  as follows. For the prior distribution  $q(\mathbf{x})$ , denote it as  $\mathbf{x} = \hat{\mathbf{x}} \oplus \delta \mathbf{x}$  with  $\delta \mathbf{x} \sim \mathcal{N}(\mathbf{0}, \hat{\mathbf{P}})$ . In case of the LiDAR update (i.e., the first step),  $(\hat{\mathbf{x}}, \hat{\mathbf{P}})$  is the state and covariance obtained from the propagation step. In case of the visual update (i.e., the second step),  $(\hat{\mathbf{x}}, \hat{\mathbf{P}})$  is the converged state and covariance obtained from the LiDAR update.

To obtain the measurement model distribution  $q(\mathbf{y} | \mathbf{x})$ , denote state estimated at the  $\kappa$ th iteration as  $\hat{\mathbf{x}}^\kappa$ , where  $\hat{\mathbf{x}}^0 = \hat{\mathbf{x}}$ . Approximating the measurement model (4) (either the LiDAR or camera measurement) through its first-order Taylor expansion made at  $\hat{\mathbf{x}}^\kappa$  leads to

$$\mathbf{y} | \mathbf{x} \simeq \underbrace{\mathbf{h}(\hat{\mathbf{x}}^\kappa, \mathbf{0})}_{\mathbf{z}^\kappa} + \mathbf{H}^\kappa \delta \mathbf{x}^\kappa + \mathbf{L}^\kappa \mathbf{v} \quad (9)$$

$$q(\mathbf{y} | \mathbf{x}) \simeq \mathcal{N}(\mathbf{h}(\hat{\mathbf{x}}^\kappa, \mathbf{0}) + \mathbf{H}^\kappa \delta \mathbf{x}^\kappa, \mathbf{R}) \quad (10)$$

where  $\delta \mathbf{x}^\kappa = \mathbf{x} \ominus \hat{\mathbf{x}}^\kappa$ ,  $\mathbf{z}^\kappa$  is the residual,  $\mathbf{L}^\kappa \mathbf{v} \sim \mathcal{N}(\mathbf{0}, \mathbf{R})$  is the lumped measurement noise,  $\mathbf{H}^\kappa$  and  $\mathbf{L}^\kappa$  are the Jacobian matrices of  $\mathbf{h}(\hat{\mathbf{x}}^\kappa \oplus \delta \mathbf{x}^\kappa, \mathbf{v})$  with respect to  $\delta \mathbf{x}^\kappa$  and  $\mathbf{v}$ , evaluated at zero, respectively.

Then, substituting the prior distribution  $q(\mathbf{x})$  and the measurement distribution  $q(\mathbf{y} | \mathbf{x})$  in (10) into the posterior distribution (8) and performing maximum likelihood estimation, we can obtain the maximum a-posterior estimation of  $\delta \mathbf{x}^\kappa$  (and hence  $\mathbf{x}^\kappa$ ) from the standard update step in the ESIKF framework [43]

$$\mathbf{K} = \left( (\mathbf{H}^\kappa)^T \mathbf{R}^{-1} \mathbf{H}^\kappa + \hat{\mathbf{P}}^{-1} \right)^{-1} (\mathbf{H}^\kappa)^T \mathbf{R}^{-1}$$

$$\hat{\mathbf{x}}^{\kappa+1} = \hat{\mathbf{x}}^\kappa \oplus (-\mathbf{K} \mathbf{z}^\kappa - (\mathbf{I} - \mathbf{K} \mathbf{H}^\kappa) (\hat{\mathbf{x}}^\kappa \ominus \hat{\mathbf{x}})). \quad (11)$$

The converged state and covariance matrix then make the mean and covariance of the posterior distribution  $q(\mathbf{x} | \mathbf{y})$ .

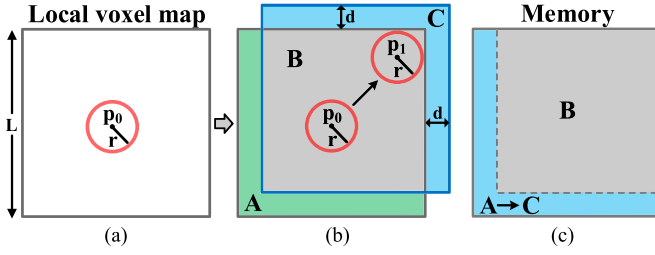


Fig. 4. 2-D demonstration of local map slide. In (a), the gray rectangle is the initial map region with length  $L$ . The red circle is the initial detection area centered at  $p_0$ . In (b), the detection area moves to a new position  $p_1$  where the map boundaries are touched. The map region is moved to a new position (blue rectangle) by distance  $d$ . In (c), the memory space  $B$  remains unchanged. The memory space  $A$  storing the green area is reset for the blue area  $C$  in (b). (a) Initial map. (b) Move the map. (c) Reset the memory.

Kalman filter with sequential update has been investigated in the literature, such as in [44] and [45]. This article adopts this approach for ESIKF for LiDAR and camera systems. The implementation of the ESIKF with sequential update is detailed in Algorithm 1. In the first step (Lines 6–10), the error state is updated from the LiDAR measurements (see Section VI-A) iteratively until convergence. The converged state and covariance estimates, denoted again as  $\hat{x}$  and  $\hat{P}$ , are used to update the geometry of the map (see Section V-B), and subsequently refined in the second step visual update (Lines 13–23) on each level of the image pyramid (see Section VII-B) until convergence. The optimal state and covariance, denoted as  $\bar{x}$  and  $\bar{P}$ , are employed for propagating incoming IMU measurements (see Section IV-C) and update the visual structures of the map (see Sections V-D and V-E).

## V. LOCAL MAPPING

### A. Map Structure

Our map employs an adaptive voxel structure presented in [14], which is organized by a Hash table and an octree for each Hash entry (see Fig. 2). The hash table manages root voxels, each with a fixed dimension of  $0.5 \times 0.5 \times 0.5$  m. Each root voxel encapsulates an octree structure to further organize leaf voxels of varying sizes. A leaf voxel represents a local plane and stores a plane feature (i.e., plane center, normal vector, and uncertainty) along with a set of LiDAR raw points situated on this plane. Some of these points are attached with three-level image patches ( $8 \times 8$  patch size), which we refer to as visual map points. Converged visual map points are only attached with reference patches, while nonconverged ones are attached with reference patches and other visible patches (see Section V-E). The varying size of the leaf voxel allows it to represent local planes of different scales, thus being adaptable to environments with different structure [14].

To prevent the size of the map from going unbound, we keep only a local map within a large local region of length  $L$  around the LiDAR's current position, as illustrated in a 2-D example in Fig. 4. Initially, the map is a cube centered at the LiDAR's starting position  $p_0$ . The LiDAR's detection area is visualized as a sphere centered at its current position, with its radius defined by the LiDAR's detection range. When the LiDAR moves to a new position  $p_1$  where the detection area touches the boundaries of the map, we move the map away from the boundaries by a

distance  $d$ . As the map moves, the memory containing the area moved out of the local map will be reset to store new areas moved in the local map. This ring-buffer approach ensures that our local map is maintained within a fixed size of memory. The implementation of the ring-buffer Hash map is detailed in [46]. The map move check is performed after each ESIKF update step.

### B. Geometry Construction and Update

The geometry of the map is constructed and updated from LiDAR point measurements. Specifically, after the LiDAR update in ESIKF (see Section IV), we register all points from the LiDAR scan to the global frame. For each registered LiDAR point, we determine its located root voxel in the Hash map. If the voxel does not exist, we initialize the voxel with the new point and index it into the Hash map. If the determined voxel already exists in the map, we append the point to the existing voxel. After all points in a scan are distributed, we conduct geometry construction and update as follows.

For newly created voxels, we determine if all the contained points lie on a plane based on the singular value decomposition. If so, we calculate the center point  $q = \bar{p}$ , plane normal  $n$ , and the covariance matrix of  $(q, n)$ , denoted as  $\Sigma_{n,q}$ , of the plane.  $\Sigma_{n,q}$  is used to characterize the plane uncertainty, which arises from both pose estimation uncertainty and the point measurement noise. The detailed plane criteria and calculation of the plane parameters and uncertainties can be referred to our previous work [14]. If the contained points do not lie on a plane, the voxel is continuously subdivided into eight smaller octants until either the points in the subvoxel are determined to form a plane or the maximum layer (e.g., 3) is reached. In the latter case, the points in the leaf voxel will be discarded. As a result, the map only contains voxels (either root or sub) identified as planes.

For existing voxels that have new points appended, we assess if the new points still form a plane with the existing points in the root voxel or subvoxel. If not, we conduct voxel subdivision as previous. If yes, we update the plane's parameters  $(q, n)$  and covariance  $\Sigma_{n,q}$  as above-mentioned too. Once the plane parameters converge (see [14]), the plane will be considered as mature and new points on this plane will be discarded. Moreover, mature planes will have their estimated plane parameters  $(q, n)$  and covariance  $\Sigma_{n,q}$  fixed.

LiDAR points on planes (either in the root voxel or subvoxel) will be used for generating visual map points in the subsequent section. For mature planes, 50 most recent LiDAR points are candidates for visual map point generation, while for immature planes; all LiDAR points are the candidates. The visual map point generation process will identify some of these candidate points as visual map points and attach them with image patches for image alignment.

### C. Visual Map Point Generation and Update

To generate and update visual map points, we select the candidate LiDAR points in the map that 1) are visible from the current frame (detailed in Section VII-A), and 2) exhibit significant gray-level gradients in the current image. We project these candidate points, after the visual update (see Section IV-D), onto the current image and retain the candidate point with the smallest depth for the local plane in each voxel. Then, we divide

the current image into uniform grid cells each with  $30 \times 30$  pixels. If a grid cell does not contain any visual map point projected here, we generate a new visual map point using the candidate point with the highest gray-level gradient and associate it with the current image patch, estimated current state (i.e., frame pose and exposure time), and the plane normal calculated from the LiDAR points as in the previous section. The patch attached to the visual map points has three layers of the same size (e.g.,  $11 \times 11$  pixels), each layer is half sampled from the previous layer, forming a patch pyramid. If a grid cell contains visual map points projected here, we add new patch (all three layers of pyramid) to the existing visual map point if 1) more than 20 frames have passed since its last patch addition, or 2) its pixel position in the current frame deviates by more than 40 pixels from its position at the last patch addition. As a result, the map points will likely have effective patches with uniformly distributed viewing angles. Along with the patch pyramid, we also attach the estimated current state (i.e., pose and exposure time) to the map point.

#### D. Reference Patch Update

A visual map point could have more than one patch due to the addition of new patches. We need to choose one reference patch for image alignment in the visual update. In detail, we score each patch  $\mathbf{f}$  based on photometric similarity and viewing angle as follows:

$$\begin{aligned} \text{NCC}(\mathbf{f}, \mathbf{g}) &= \frac{\sum_{x,y} [\mathbf{f}(x,y) - \bar{\mathbf{f}}] [\mathbf{g}(x,y) - \bar{\mathbf{g}}]}{\sqrt{\sum_{x,y} [\mathbf{f}(x,y) - \bar{\mathbf{f}}]^2 \sum_{x,y} [\mathbf{g}(x,y) - \bar{\mathbf{g}}]^2}} \\ c &= \frac{\mathbf{n} \cdot \mathbf{p}}{\|\mathbf{p}\|}, \quad \omega_1 = \frac{1}{1 + e^{\text{tr}(\Sigma_{\mathbf{n}})}} \\ S &= (1 - \omega_1) \cdot \frac{1}{n} \sum_{i=1}^n \text{NCC}(\mathbf{f}, \mathbf{g}_i) + \omega_1 \cdot c \end{aligned} \quad (12)$$

where  $\text{NCC}(\mathbf{f}, \mathbf{g})$  represents the normalized cross-correlation (NCC) used to measure the similarity between patch  $\mathbf{f}$  and  $\mathbf{g}$  at the 0th pyramid level (the level with the highest resolution) of both patch, with mean subtraction applied to both patches,  $c$  denotes the cosine similarity between the normal vector  $\mathbf{n}$  and view direction  $\mathbf{p}/\|\mathbf{p}\|$  of patch  $\mathbf{f}$  under evaluation. When the patch is directly facing the plane where the map point is located, the value of  $c$  is 1. The overall score  $S$  is calculated by summing the weighted NCC and  $c$ , where the former represents the average similarity between the patch  $\mathbf{f}$  under evaluation and all other patches  $\mathbf{g}_i$  and  $\text{tr}(\Sigma_{\mathbf{n}})$  represents the trace of the covariance matrix of the normal vector.

Among all the patches attached to a visual map point, the one with the highest score is updated as the reference patch. The above-mentioned scoring mechanism tends to choose reference patches whose 1) appearance is similar (in terms of NCC) to most of the rest of the patches, a technique used by MVS [47] to avoid patches on dynamic objects; 2) view direction is orthogonal to the plane, thereby maintaining texture details at a high resolution. In contrast, the reference patches update strategy in our previous work FAST-LIVO [8] and prior arts [4] directly select the patch with the smallest view direction difference from the current frame, causing the selected reference patch to be very close to the current frame, hence imposing weak constraints on the current pose update.

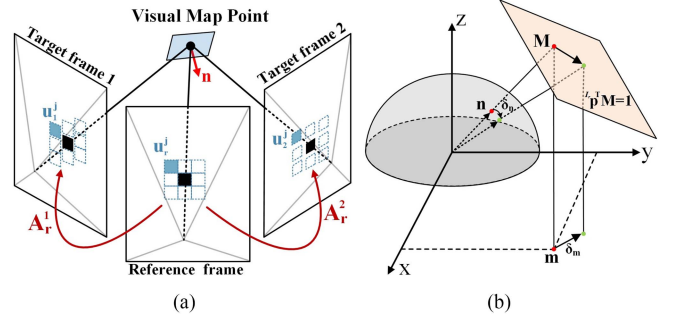


Fig. 5. (a) Affine warping between reference patches and target patches. (b) Any normal  ${}^I_r \mathbf{n} \in \mathbb{S}^2$  lying on the normalized sphere is first projected into a point  $\mathbf{M} \in \mathbb{R}^3$  on the plane  ${}^I_r \mathbf{p}^T \mathbf{M} = 1$ , and then projected to a point  $\mathbf{m} \in \mathbb{R}^2$  on the  $x$ - $y$  plane. This transformation thereby converts a perturbation  $\delta \mathbf{n}$  on the sphere into a perturbation  $\delta \mathbf{m}$  in  $x$ - $y$  plane.

#### E. Normal Refine

Each visual map point is assumed to lie on a small local plane. Existing works [2], [4], [8] assumed that all pixels in a patch have the same depth, a wild assumption that does not hold in general. We use plane parameters computed from the LiDAR points as detailed in Section V-B to achieve greater accuracy. This plane normal is crucial for performing affine warping for image alignment in the visual update process. To further enhance the accuracy of the affine warping, the plane normal could be further refined from the patches attached to the visual map point. Specifically, we refine the plane normal in the reference patch by minimizing the photometric error with respect to the other patches attached to the visual map point.

1) *Affine Warping*: Affine warping is used to transform patch pixels from the reference frame (i.e., the source patch) to patch pixels in the rest of the frames (i.e., the target patch), illustrated in Fig. 5(a). Let  $\mathbf{u}_r^j$  be the  $j$ th pixel coordinates in the source patch and  $\mathbf{u}_i^j$  be the  $j$ th pixel coordinates in the  $i$ th target patch. Assuming all pixels in the patch lie in a local plane with normal  ${}^I_r \mathbf{n}$  and visual map point position  ${}^I_r \mathbf{p}$  (which corresponds to the center pixel for both source and target patches), both represented in the source patch frame, we have

$$\begin{aligned} \mathbf{u}_i^j &= \mathbf{A}_r^i \mathbf{u}_r^j \\ \mathbf{A}_r^i &= \mathbf{P} \left( {}^I_i \mathbf{R}_{I_r} + {}^I_i \mathbf{t}_{I_r} \frac{1}{{}^I_r \mathbf{n}^T \cdot {}^I_r \mathbf{p}} {}^I_r \mathbf{n}^T \right) \mathbf{P}^{-1} \end{aligned} \quad (13)$$

where  $\mathbf{A}_r^i$  represents the affine warping matrix that transforms the pixel coordinates from the source (or reference) patch to the  $i$ th target patch,  ${}^I_i \mathbf{R}_{I_r}$  and  ${}^I_i \mathbf{t}_{I_r}$  denote the relative pose of the reference frame  $I_r$  w.r.t. the target frame  $I_i$ . To use fisheye images directly without rectifying them to pinhole images, we implement projection matrix  $\mathbf{P}$  and back projection matrix  $\mathbf{P}^{-1}$  based on different camera models (e.g.,  $\mathbf{P}$  is the camera intrinsic matrix for the pinhole camera model).

2) *Normal Optimization*: To refine the plane normal  ${}^I_r \mathbf{n}$ , we minimize photometric errors between the reference patch and other image patches at the 0th pyramid level (i.e., the highest resolution level)

$${}^I_r \mathbf{n}^* = \arg \min_{{}^I_r \mathbf{n} \in \mathbb{S}^2} \sum_{i \in S} \sum_{j=1}^{N^2} \left\| \tau_i \mathbf{I}_i (\mathbf{A}_r^i \mathbf{u}_r^j) - \tau_r \mathbf{I}_r (\mathbf{u}_r^j) \right\|_2 \quad (14)$$

where  $N$  is the path size,  $\tau_r$  and  $\tau_i$  are the inverse exposure times of the reference frame and the  $i$ th target frame, respectively.  $\mathbf{I}_r(\mathbf{u}_r^j)$  denote  $j$ th patch pixel in the reference frame,  $\mathbf{I}_i(\mathbf{A}_r^i \mathbf{u}_r^j)$  denotes the  $j$ th patch pixel in the  $i$ th target frame, and  $S$  is the set of all target frames.

3) *Optimization Variable Transformation*: To enhance the computational efficiency, we reparameterize the least squares problem in (14). Note that the optimization variable  ${}^I_r \mathbf{n}$  only appeared in  $\mathbf{M} \triangleq \frac{1}{{}^I_r \mathbf{n}^T \cdot {}^I_r \mathbf{p}} {}^I_r \mathbf{n} \in \mathbb{R}^3$  in (13), the optimization over  ${}^I_r \mathbf{n}$  can be conducted over  $\mathbf{M}$ . Moreover, the vector  $\mathbf{M}$  is subject to constraint  ${}^I_r \mathbf{p} \cdot \mathbf{M} = 1$ , meaning that  $\mathbf{M}$  can be parameterized as follows:

$$\mathbf{M} = \begin{bmatrix} \mathbf{M}_x \\ \mathbf{M}_y \\ \frac{1}{{}^I_r \mathbf{p}_z} - \frac{{}^I_r \mathbf{p}_x}{{}^I_r \mathbf{p}_z} \mathbf{M}_x - \frac{{}^I_r \mathbf{p}_y}{{}^I_r \mathbf{p}_z} \mathbf{M}_y \end{bmatrix} = \mathbf{B} \mathbf{m} + \mathbf{b}$$

$$\mathbf{B} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ -\frac{{}^I_r \mathbf{p}_x}{{}^I_r \mathbf{p}_z} & -\frac{{}^I_r \mathbf{p}_y}{{}^I_r \mathbf{p}_z} \end{bmatrix}, \mathbf{b} = \begin{bmatrix} 0 \\ 0 \\ \frac{1}{{}^I_r \mathbf{p}_z} \end{bmatrix}, \mathbf{m} = \begin{bmatrix} \mathbf{M}_x \\ \mathbf{M}_y \end{bmatrix} \in \mathbb{R}^2 \quad (15)$$

where  ${}^I_r \mathbf{p}_z \neq 0$  since no such reference patch could be chosen for the visual map point. The relation among  ${}^I_r \mathbf{n}$ ,  $\mathbf{M}$ , and  $\mathbf{m}$  are shown in Fig. 5(b).

Finally, the optimization in (14) is conducted over the vector  $\mathbf{m} \in \mathbb{R}^2$  without any constraints. This optimization can be performed in a separate thread to avoid blocking the main odometry thread. The optimized parameter  $\mathbf{m}^*$  can then be used to recover the optimal normal vector  ${}^I_r \mathbf{n}^*$

$${}^I_r \mathbf{n}^* = \frac{\mathbf{M}^*}{\|\mathbf{M}^*\|}, \mathbf{M}^* = \mathbf{B} \mathbf{m}^* + \mathbf{b}. \quad (16)$$

Once the plane normal converges, the reference patch and normal vector for this visual map point are fixed without further refinement, and all other patches are deleted.

## VI. LiDAR MEASUREMENT MODEL

This section details the LiDAR measurement model  $\mathbf{y}_l = \mathbf{h}_l(\mathbf{x}, \mathbf{v}_l)$  used in the LiDAR update of ESIKF in Section IV-D.

### A. Point-to-Plane LiDAR Measurement Model

After obtaining the undistorted points  $\{{}^L \mathbf{p}_j\}$  in a scan, we project them to the global frame using the estimated state  $\hat{\mathbf{x}}^\kappa$  at the  $\kappa$ th iteration of LiDAR update

$${}^G \hat{\mathbf{p}}_j^\kappa = {}^G \hat{\mathbf{T}}_I^\kappa {}^L \mathbf{p}_j. \quad (17)$$

We then identify the root or subvoxel where  ${}^G \hat{\mathbf{p}}_j^\kappa$  lies in the Hash map. If no voxel is found or the voxel does not contain a plane, the point is discarded. Otherwise, we use the plane in the voxel to establish a measurement equation for the LiDAR point. Specifically, we assume the true LiDAR point  ${}^L \mathbf{p}_j^{gt}$ , given the accurate LiDAR pose  ${}^G \mathbf{T}_I$ , should lie on the plane with normal  $\mathbf{n}_j^{gt}$  and center point  $\mathbf{q}_j^{gt}$  in the voxel. i.e.,

$$\mathbf{0} = (\mathbf{n}_j^{gt})^T ({}^G \mathbf{T}_I {}^L \mathbf{p}_j^{gt} - \mathbf{q}_j^{gt}). \quad (18)$$

Since the ground-true point  ${}^L \mathbf{p}_j^{gt}$  is measured as  ${}^L \mathbf{p}_j$  with ranging and bearing noises  $\delta {}^L \mathbf{p}_j$ , we have  ${}^L \mathbf{p}_j^{gt} = {}^L \mathbf{p}_j - \delta {}^L \mathbf{p}_j$ . Likewise, the plane parameters  $(\mathbf{n}_j^{gt}, \mathbf{q}_j^{gt})$  are estimated as

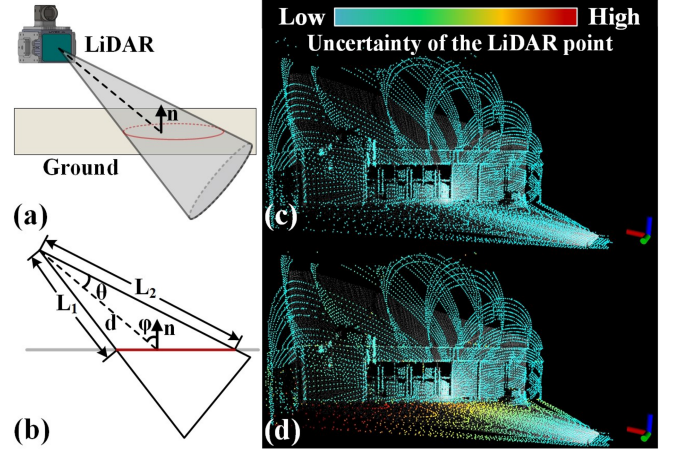


Fig. 6. (a) and (b), respectively, illustrate the 3-D and side crosssectional views of the LiDAR point uncertainty model considering the laser beam divergence angle  $\theta$ . The red contour outlines the area that a laser beam spreads. (c) and (d) color the points in a scan by the point location uncertainty. Compared to (c), (d) further takes into account the ranging uncertainty  $\delta d$  resulting from the beam divergence angle. This leads to a higher uncertainty for ground points due to the large spread area of laser beams.

$(\mathbf{n}_j, \mathbf{q}_j)$  with covariance  $\Sigma_{\mathbf{n}, \mathbf{q}}$  (see Section V-B), so we have:  $\mathbf{n}_j^{gt} = \mathbf{n}_j \boxminus \delta \mathbf{n}_j$ ,  $\mathbf{q}_j^{gt} = \mathbf{q}_j - \delta \mathbf{q}_j$ . Therefore

$$\underbrace{\mathbf{0}}_{\mathbf{y}_l} = (\mathbf{n}_j \boxminus \delta \mathbf{n}_j)^T \underbrace{({}^G \mathbf{T}_I {}^L \mathbf{p}_j - \delta {}^L \mathbf{p}_j - (\mathbf{q}_j - \delta \mathbf{q}_j))}_{\mathbf{h}_l(\mathbf{x}, \mathbf{v}_l)} \quad (19)$$

where the measurement noise  $\mathbf{v}_l = (\delta {}^L \mathbf{p}_j, \delta \mathbf{n}_j, \delta \mathbf{q}_j)$  consisting of the noise associated with the LiDAR point, the normal vector, and the plane center, respectively.

### B. LiDAR Measurement Noise With Beam Divergence

The uncertainty of a LiDAR point  $\delta {}^L \mathbf{p}_j$  in the local LiDAR frame is decomposed into two components in [14], the ranging uncertainty  $\delta d$  caused by laser time of flight (TOF), and the bearing direction uncertainty  $\delta \omega$  originated from encoders. Besides these uncertainties, we also consider uncertainties caused by the laser beam divergence angle  $\theta$ , as illustrated in Fig. 6. As the angle  $\varphi$  between the bearing direction and normal vector increases, the ranging uncertainty of the LiDAR point increases significantly, while the bearing direction uncertainty remains unaffected. The  $\delta d$  due to the laser beam divergence angle can be modeled as

$$\delta d = L_2 - L_1 = d \left( \frac{\cos \varphi}{\cos(\theta + \varphi)} - \frac{\cos \varphi}{\cos(\theta - \varphi)} \right). \quad (20)$$

Considering  $\delta d$  influenced by TOF and laser beam divergence, when our system selects more points from the ground or walls [see Fig. 6(c) and (d)], it achieves a more precise pose estimation than that not considering such effect.

## VII. VISUAL MEASUREMENT MODEL

This section details the visual measurement model  $\mathbf{y}_c = \mathbf{h}_c(\mathbf{x}, \mathbf{v}_c)$  used in the visual update of ESIKF in Section IV-D.

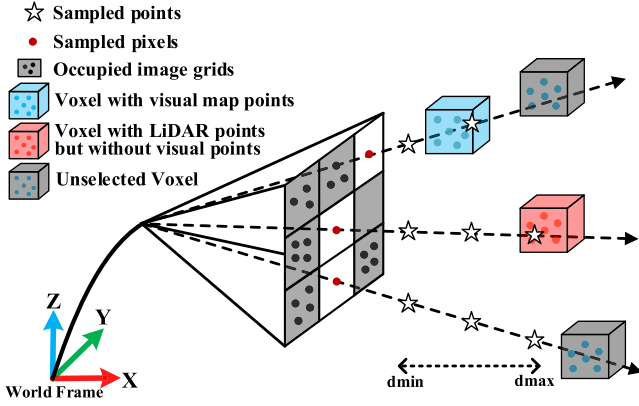


Fig. 7. Illustration of on-demand voxel raycasting.

### A. Visual Map Point Selection

To perform sparse image alignment in the visual update, we begin by selecting appropriate visual map points. We first extract the set of map points (termed as visual submap) that is visible in the current camera FoV, using voxel, and raycasting queries. Then, the visual map points from this submap are selected and outliers are rejected. This process yields a refined set of visual map points ready for constructing visual photometric errors in the visual measurement model.

1) *Visible Voxel Query*: Identifying map voxels within the current frame FoV is challenging due to the large number of voxels in the map. To address this issue, we poll the voxels hit by LiDAR points in the current scan. This can be done efficiently by inquiring the voxel Hash table using the measured point position. If the camera FoV is largely overlapped with the LiDAR FoV, map points in the camera FoV likely lie in these voxels as well. We also poll voxels hit by map points identified visible (through the same voxel query and raycasting) in the previous image frame, assuming that two consecutive image frames have large FoV overlaps. Finally, the current visual submap can be obtained as map points contained in these two types of voxels followed by an FoV check.

2) *Raycasting on Demand*: In most cases, the visual submap can be obtained through the voxel queries above. However, a LiDAR sensor could return no points when it is too close to an object (known as the close proximity blind zones). Also, the camera FoV may not be completely covered by the LiDAR FoV. To recall more visual map points in these cases, we employ a raycasting strategy, as illustrated in Fig. 7. We divide the image into uniform grid cells, each with  $30 \times 30$  pixels, and project the visual map points obtained from the voxel query onto the grid cells. For each image grid cell that is not occupied by these visual map points, a ray is cast backward along the central pixel, where sample points are uniformly distributed along the ray in the depth direction from  $d_{\min}$  to  $d_{\max}$ . In order to reduce computation load, the positions of sample points on each ray in the camera body frame are precomputed. For each sampled point, we evaluate the corresponding voxel's status: if the voxel contains map points that lie in this grid cell after projection, we incorporate these map points into the visual submap and cease for this ray. Otherwise, we continue to the next sample point on the ray until reaching the maximum depth  $d_{\max}$ . After processing the entire unoccupied image grid cells

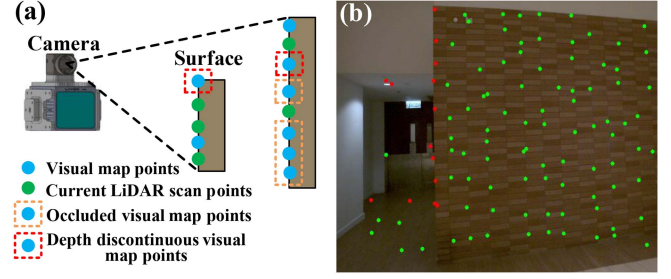


Fig. 8. Outlier rejection. (a) shows the diagrammatic drawing of occluded and depth-discontinuous visual map points. (b) shows the effect of outlier rejection in real scenes. The red dots are the rejected visual map points, and the green dots are the accepted visual map points.

through raycasting, we obtain a set of visual map points that distribute among the whole image.

3) *Outlier Rejection*: After voxel query and raycasting, we obtain all visual map points in the current frame FoV. However, these visual map points could be occluded in the current frame, have discontinuous depth, have their reference patch taken at large view angles, or have large view angles in the current frame, all of which can severely degrade image alignment accuracy. To address the first issue, we project all the visual map points in the submap into the current frame using the pose after the LiDAR update and keep the lowest depth points in each grid cell of  $30 \times 30$  pixels. To address the second issue, we project the LiDAR points in the current LiDAR scan to the current frame producing a depth map. By comparing the depth of visual map points with their  $9 \times 9$  neighbor in the depth map, we determine their occlusion and depth variation. Occluded and depth-discontinuous map points are rejected (see Fig. 8). To address the third and fourth issue, we remove points where the view angle (i.e., the angle between normal vector and direction from the visual map point to the patch optical center) of the reference patch or current patch is too large (e.g., over  $80^\circ$ ). The remaining visual map points will be used to align the current image.

### B. Sparse-Direct Visual Measurement Model

The visual map points  $\{^G\mathbf{p}_i\}$  extracted previously are used to construct the visual measurement model. The underlying principle is that, when transforming the map point  $^G\mathbf{p}_i$  to the current image  $\mathbf{I}_k(\cdot)$  with the ground-truth state (i.e., pose)  $\mathbf{x}_k$ , the photometric error between the reference patch and the current patch should be zero

$$\begin{aligned} 0 = & \tau_k \mathbf{I}_k^{gt} \left( \underbrace{\pi \left( {}^C\mathbf{T}_I ({}^G\mathbf{T}_I)^{-1} {}^G\mathbf{p}_i \right)}_{\mathbf{u}_i} + \Delta \mathbf{u} \right) \\ & - \tau_r \mathbf{I}_r^{gt} \left( \underbrace{\pi \left( {}^{C_r}\mathbf{T}_G {}^G\mathbf{p}_i \right)}_{\mathbf{u}'_i} + \mathbf{A}_i^r \Delta \mathbf{u} \right) \end{aligned} \quad (21)$$

where  $\pi(\cdot)$  is the common camera projection model (i.e., Pin-hole, MEI, ATAN, Scaramuzza, Equidistant),  ${}^{C_r}\mathbf{T}_G$  is the pose of the global frame  $G$  w.r.t. reference frame  $C_r$ , which has been estimated when receiving and fusing the reference frame,  $\mathbf{A}_i^r$  is the affine warping matrix that transforms pixels from the  $i$ th current patch to the reference patch,  $\Delta \mathbf{u}$  is the relative pixel position to the center  $\mathbf{u}_i$  within the current patch,  $\mathbf{I}_k^{gt}$ ,  $\mathbf{I}_r^{gt}$  denote the ground-truth pixel values of the reference and current frames,

respectively. They are measured as the actual image pixel values  $\mathbf{I}_k, \mathbf{I}_r$  with measurement noise  $\mathbf{v}_c = (\delta\mathbf{I}_k, \delta\mathbf{I}_r)$ , which originate from various sources [e.g., shot noise and the analog-to-digital converter (ADC) noise of the camera CMOS]. Hence

$$\underbrace{\mathbf{0}}_{\mathbf{y}_c} = \underbrace{\tau_k (\mathbf{I}_k (\mathbf{u}_i + \Delta\mathbf{u}) - \delta\mathbf{I}_k) - \tau_r (\mathbf{I}_r (\mathbf{u}'_i + \mathbf{A}_i^T \Delta\mathbf{u}) - \delta\mathbf{I}_r)}_{\mathbf{h}_c(\mathbf{x}, \mathbf{v}_c)}. \quad (22)$$

To enhance computational efficiency, we employ an inverse compositional formulation [4], [48], where the pose incremental  $\delta\mathbf{T} \in \mathbb{R}^6$  parameterizing  ${}^G\mathbf{T}_I = {}^G\hat{\mathbf{T}}_I^\kappa \text{Exp}(\delta\mathbf{T})$  in  $\mathbf{u}_i$  [see (21)], is moved from  $\mathbf{u}_i$  to  $\mathbf{u}'_i$  as follows:

$$\begin{aligned} \mathbf{u}_i &= \pi \left( {}^C\mathbf{T}_I \left( {}^G\hat{\mathbf{T}}_I^\kappa \right)^{-1} {}^G\mathbf{p}_i \right) \\ \mathbf{u}'_i &= \pi \left( {}^C\mathbf{T}_G \text{Exp}(\delta\mathbf{T}) {}^G\mathbf{p}_i \right). \end{aligned} \quad (23)$$

Given that  $\mathbf{u}'_i$  in the reference frame remains unchanged during each iteration, we only require a one-time computation of the Jacobian matrices w.r.t.  $\delta\mathbf{T}$ , rather than recalculating them for every iteration.

To estimate the inverse exposure time  $\tau_k$  from the measurement (22), we fix the initial inverse exposure time  $\tau_0 = 1$  to eliminate the degeneration of (22) when all inverse exposure time are zeros. The estimated inverse exposure times of subsequent frames are therefore the exposure time relative to the first frame.

Equation (22) is used in the visual update step across three levels (see Algorithm 1); the visual update starts from the coarsest level, after the convergence of a level, it proceeds to the next finer level. The estimated state is then used to generate visual map points (see Section V-C) and update reference patch (see Section V-D).

### VIII. DATASETS FOR EVALUATION

In this section, we introduce datasets for performance evaluation, including public datasets NTU-VIRAL [49], Hilti'22 [50], Hilti'23 [51], and MARS-LVIG [52], as well as our self-collected FAST-LIVO2 private dataset. Specifically, the NTU-VIRAL and Hilti datasets are used to conduct a quantitative benchmark comparison of our system against other SOTA SLAM systems (see Section IX-B). The FAST-LIVO2 private dataset is primarily used to evaluate our system across various extremely challenging scenarios (see Section IX-C), to demonstrate its capability for high-precision mapping (see Section IX-D), and to validate the functionality of the individual modules within our system (see Sections I-A through I-D in the Supplementary Material [53]). MARS-LVIG dataset is employed for application demonstrations (see Section X) and ablation study (see Section I-E in the Supplementary Material [53]).

#### A. NTU-VIRAL, Hilti, and MARS-LVIG Dataset

The NTU-VIRAL dataset, collected at the Nanyang Technological University campus using an aerial platform, presents diverse scenarios embodying unique aerial operational challenges. Specifically, the “sbs” sequences can only provide noisy visual features from distant objects. The “nya” sequences present challenges to LiDAR SLAM due to semitransparent surfaces and to visual SLAM owing to intricate flight dynamics and low lighting conditions. The dataset is equipped with a 16-channel

OS1 gen1<sup>3</sup> LiDAR sampled at 10 Hz and with a built-in IMU at 100 Hz, and two synchronized pinhole cameras triggered at 10 Hz. The left camera is used for evaluation.

The Hilti'22 and Hilti'23 datasets, collected by handheld and robot devices, encompass indoor and outdoor sequences from environments, such as construction sites, offices, labs, and parking areas. These sequences introduce numerous challenges from long corridors, basements, and stairs, with textureless features, varying illumination conditions, and insufficient LiDAR plane constraints. Handheld sequences use a Hesai PandarXT-32<sup>4</sup> LiDAR at 10 Hz, five wide-angle cameras at 40 Hz, which is downsampled into 10 Hz, and an external Bosch BMI085 IMU at 400 Hz. Meanwhile, the robot-mounted sequences feature a Robosense Bpearl<sup>5</sup> LiDAR at 10 Hz, eight omnidirectional cameras at 10 Hz, and an Xsens MTi-670 IMU at 200 Hz. In both cases, the front-facing camera is used for all systems under evaluation. Millimeter-accurate ground truth, obtained through a motion capture system or a Total Station [54], is provided for each sequence. Note that the ground truth of the Hilti datasets is not open-source; therefore, algorithmic results on these datasets are evaluated via the Hilti official website. Since “Site 3” in Hilti'23 does not provide in-depth analysis plots (e.g., RMSE), we exclude these four sequences, but our scoring results for these sequences can still be found on their official website.<sup>6</sup> NTU-VIRAL and Hilti contribute a total of 25 sequences.

The MARS-LVIG dataset provides high-altitude, ground-facing mapping data that encompasses diverse unstructured terrains, such as jungles, mountains, and islands. The dataset was collected via a DJI M300 RTK quadrotor, which is equipped with a Livox Avia<sup>7</sup> LiDAR (with built-in BMI088 IMU) and a high-resolution global-shutter camera, both triggered at 10 Hz. This is notably distinct from the aforementioned NTU-VIRAL and Hilti datasets, which use  $752 \times 480$  grayscale images, while the MARS dataset employs  $2448 \times 2048$  RGB images, thereby facilitating the generation of clear, dense colored point clouds. Therefore, we leverage this public dataset to validate our capabilities in high-altitude aerial mapping applications.

#### B. FAST-LIVO2 Private Dataset

To validate the system's performance under more extreme conditions (e.g., LiDAR degeneration, low illumination, drastic exposure changes, and cases of no LiDAR measurements), we make a new dataset named FAST-LIVO2 private dataset. The dataset, hardware device, and hardware synchronization scheme are released with the codes of this work to facilitate the reproduction of our work.

1) *Platform*: Our data collection platform, illustrated in Fig. 9, is equipped with an industrial camera (MV-CA013-21UC), a Livox Avia LiDAR, and a DJI manifold-2c (Intel i7-8550u CPU and 8GB RAM) as onboard computers. The camera FoV is  $70.6^\circ \times 68.5^\circ$  and the LiDAR FoV is  $70.4^\circ \times 77.2^\circ$ . All sensors are hard synchronized with a 10 Hz trigger signal, generated by STM32 synchronized timers.

2) *Sequence Description*: As summarized in Table S1 in the Supplementary Material [53], the FAST-LIVO2 dataset

<sup>3</sup>[Online]. Available: <https://ouster.com/products/os1-lidar-sensor>

<sup>4</sup>[Online]. Available: <https://www.hesaitech.com/product/xt32/>

<sup>5</sup>[Online]. Available: <https://www.robosense.ai/en/rslidar/RS-Bpearl>

<sup>6</sup>[Online]. Available: <https://hilti-challenge.com/leader-board-2022.html>, <https://hilti-challenge.com/leader-board-2023.html>

<sup>7</sup>[Online]. Available: <https://www.livoxtech.com/avia>

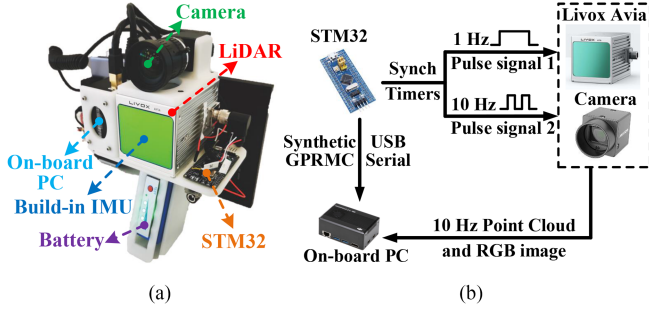


Fig. 9. Our platform with hardware synchronization for data acquisition. (a) Our handheld platform. (b) Hardware synchronization scheme.

comprises 20 sequences across various scenes (e.g., campus buildings, corridors, basements, mining tunnel, etc.) characterized by structure-less, cluttered, dim, variable-lighting, and weakly textured environments, with a total duration of 66.9 min. Most sequences exhibit visual and/or LiDAR degeneration, such as facing a single and/or texture-less plane, traversing an extremely narrow and/or dark tunnel, and experiencing varying light conditions from indoor to outdoor (see Fig. S7 in the Supplementary Material [53]). To guarantee enhanced synchronous data collection between the camera and LiDAR, we configure the camera with fixed exposure time but autogain mode in most scenarios. For the remaining sequences with autoexposure, we record their ground truth exposure times. In all sequences, the platform returns to the starting point, which enables the drift evaluation.

## IX. EXPERIMENT RESULTS

In this section, we conduct extensive experiments to evaluate our proposed system.

### A. Implementation and System Configurations

We implemented the proposed FAST-LIVO2 system in C++ and robots operating system. In the default configuration, the exposure time estimation is enabled, while normal vector refinement is turned OFF. LiDAR points in a scan are downsampled temporally at a 1:3 ratio. The root voxel size for the voxel map is set at 0.5 m, and the maximum layer of the internal octree is 3. The image patch size is  $8 \times 8$  for image alignment and  $11 \times 11$  for normal refinement. Within the sequential ESIKF settings, for all the experiments, the camera photometric noise is set to a constant value of 100. The LiDAR depth error and bearing angle error are adjusted to 0.02 m and  $0.05^\circ$  for Livox Avia LiDAR and OS1-16, 0.001 m and  $0.001^\circ$  for PandarXT-32, 0.008 m and  $0.01^\circ$  for Robosense BPearl LiDAR. The laser beam divergence angle is set at  $0.15^\circ$  for Livox Avia LiDAR and OS1-16, and at  $0.001^\circ$  for the PandarXT-32 and Robosense BPearl LiDAR. Our system uses the same parameters in all sequences of all datasets with the same sensor setup. The computation platform for all experiments is a desktop PC equipped with an Intel i7-10700K CPU and 32GB RAM. For FAST-LIVO2, we also test it on an ARM processor that is commonly used in embedded systems with reduced power and cost. The ARM platform is RB5<sup>8</sup> with a Qualcomm Kryo585 CPU and 8GB RAM. We refer to the

implementation of FAST-LIVO2 on the ARM-based platform as “FAST-LIVO2 (ARM).”

### B. Benchmark Experiments

In this experiment, we conduct quantitative evaluations on 25 sequences from the NTU-VIRAL, Hilti’22, and 23 open datasets. Our approach is benchmarked against several SOTA open-source odometry systems, including R3LIVE [36], a dense direct LIVO system; FAST-LIO2 [13], a direct LiDAR-inertial odometry system; SDV-LOAM [40], a semidirect LiDAR-visual odometry system; LVI-SAM [35], a feature-based LiDAR-inertial-visual SLAM system; and our previous work FAST-LIVO [8].

These systems are downloaded from their respective GitHub repositories. For FAST-LIO2, FAST-LIVO, and LVI-SAM, we use the recommended settings for indoor and outdoor scenes equipped with multiline LiDAR sensors. For R3LIVE, we adapt the system to work with fisheye camera models and multiline LiDARs equipped with external IMUs (the default configuration only supports internal IMUs). We disable the real-time optimization of the camera intrinsic and the extrinsic  ${}^C T_I$  due to adverse optimization caused by insufficient IMU excitation in the datasets. Other parameters, including the window size and pyramid level for optical flow tracking, the resolution for downsampling the point cloud of the current scan and the global map, are fine-tuned to achieve optimal performance. Since only the vision module of SDV-LOAM is open-sourced, we integrate it with LeGO-LOAM [7] in a loosely coupled manner, following the methodology described in the original paper [40]. This enhanced system continues to refine poses obtained from the vision module and we also open this implementation on GitHub.<sup>9</sup> Given that all compared systems are odometry without loop closure, except for LVI-SAM, we remove the loop-closure module of LVI-SAM to ensure a fair comparison. In addition, we conduct an ablation study on the exposure time estimation module, the normal refine module, and the reference patch update strategy. The default FAST-LIVO2 has real-time exposure estimation and reference patch update, but no normal refinement.

The results of all methods are shown in Table II. It is seen that our method achieves the highest overall accuracy across all sequences with an average RMSE of 0.044 m, which is three times more accurate than the second-place FAST-LIVO at 0.137 m. Our system delivers the best results in most sequences, except for “Outside Building” and “Large Room (dark),” where our system exhibits a slightly (millimeter-level) higher error compared to the LiDAR-inertial only odometry FAST-LIO2. This discrepancy can be attributed to the rich structural features but poor lighting conditions of these sequences, resulting in dim and blurred images. Consequently, fusing these low-quality images does not enhance odometry accuracy. Excluding these two sequences, our method, which leverages tightly coupled LiDAR, inertial, and visual information, outperforms FAST-LIO2, our LIO subsystem, and the LiDAR-visual only odometry, SDV-LOAM, significantly. Notably, SDV-LOAM performs particularly poorly on the Hilti datasets due to its lack of tight integration with IMU measurements, leading to drift in the LO subsystem. In addition, the loose coupling between LiDAR and visual observations, along with poor initial values for

<sup>8</sup>[Online]. Available: [https://www.adlinktech.com/Products/Computer\\_on\\_Modules/SMARC/LEC-RB5?lang=en](https://www.adlinktech.com/Products/Computer_on_Modules/SMARC/LEC-RB5?lang=en)

<sup>9</sup>[Online]. Available: [https://github.com/xuankuzcr/SDV-LOAM\\_reimplementation](https://github.com/xuankuzcr/SDV-LOAM_reimplementation)

TABLE II  
ABSOLUTE TRANSLATIONAL ERRORS (RMSE, METERS) IN SEQUENCES

| Dataset   | Sequence                | SDV-LOAM | Our LIO | FAST-LIO2    | R3LIVE | LVI-SAM | FAST-LIVO | Ours (w/o expo) | Ours (w normal) | Ours (w/o update) | Ours         |
|-----------|-------------------------|----------|---------|--------------|--------|---------|-----------|-----------------|-----------------|-------------------|--------------|
| Hilti'22  | Construction ground     | 25.121   | 0.011   | 0.013        | 0.021  | ×       | 0.022     | 0.011           | <b>0.008</b>    | 0.015             | 0.010        |
|           | Construction multilevel | 12.561   | 0.031   | 0.044        | 0.024  | ×       | 0.052     | 0.021           | <b>0.018</b>    | 0.025             | 0.020        |
|           | Construction stairs     | 9.212    | 0.221   | 0.320        | 0.784  | 9.142   | 0.241     | 0.049           | 0.027           | 0.151             | <b>0.016</b> |
|           | Long corridor           | 19.531   | 0.061   | 0.064        | 0.061  | 6.312   | 0.065     | 0.069           | <b>0.059</b>    | 0.071             | 0.067        |
|           | Cupola                  | 9.321    | 0.221   | 0.250        | 2.142  | ×       | 0.182     | 0.161           | 0.122           | 0.179             | <b>0.121</b> |
|           | Lower gallery           | 11.232   | 0.014   | 0.024        | 0.008  | 2.281   | 0.022     | 0.010           | 0.008           | 0.010             | <b>0.007</b> |
|           | Attic to upper gallery  | 4.551    | 0.223   | 0.720        | 2.412  | ×       | 0.621     | 0.101           | 0.077           | 0.221             | <b>0.069</b> |
|           | Outside building        | 2.622    | 0.030   | <b>0.028</b> | 0.029  | 0.952   | 0.052     | 0.042           | 0.033           | 0.050             | 0.035        |
| Hilti'23  | Floor 0                 | 4.621    | 0.028   | 0.031        | 0.022  | ×       | 0.021     | 0.025           | 0.023           | 0.023             | <b>0.018</b> |
|           | Floor 1                 | 7.951    | 0.025   | 0.031        | 0.024  | 8.682   | 0.022     | 0.024           | <b>0.022</b>    | 0.031             | 0.023        |
|           | Floor 2                 | 7.912    | 0.041   | 0.083        | 0.046  | ×       | 0.048     | 0.023           | <b>0.021</b>    | 0.051             | 0.022        |
|           | Basement                | 6.151    | 0.021   | 0.038        | 0.024  | ×       | 0.035     | 0.020           | 0.018           | 0.018             | <b>0.016</b> |
|           | Stairs                  | 9.032    | 0.110   | 0.170        | 0.110  | 3.584   | 0.152     | 0.025           | 0.020           | 0.132             | <b>0.018</b> |
|           | Parking 3x floors down  | 19.952   | 0.162   | 0.320        | 0.462  | ×       | 0.356     | 0.035           | <b>0.022</b>    | 0.112             | 0.032        |
|           | Large room              | 16.781   | 0.121   | 0.028        | 0.035  | 0.563   | 0.031     | 0.033           | 0.027           | 0.118             | <b>0.026</b> |
|           | Large room (dark)       | 15.012   | 0.051   | <b>0.040</b> | 0.059  | ×       | 0.053     | 0.049           | 0.051           | 0.058             | 0.046        |
| NTU VIRAL | eee_01                  | 0.301    | 0.122   | 0.212        | 0.072  | 3.901   | 0.191     | 0.069           | <b>0.066</b>    | 0.109             | 0.068        |
|           | eee_02                  | 1.842    | 0.131   | 0.172        | 0.059  | 0.182   | 0.132     | 0.051           | 0.055           | 0.112             | <b>0.051</b> |
|           | eee_03                  | 0.301    | 0.124   | 0.213        | 0.078  | 0.287   | 0.192     | 0.068           | 0.070           | 0.099             | <b>0.068</b> |
|           | nya_01                  | 0.202    | 0.084   | 0.141        | 0.080  | 0.205   | 0.121     | 0.075           | 0.078           | 0.106             | <b>0.073</b> |
|           | nya_02                  | 0.214    | 0.153   | 0.212        | 0.084  | 1.296   | 0.182     | 0.076           | 0.081           | 0.118             | <b>0.075</b> |
|           | nya_03                  | 0.251    | 0.082   | 0.133        | 0.079  | 0.176   | 0.112     | 0.060           | 0.060           | 0.092             | <b>0.059</b> |
|           | sbs_01                  | 0.212    | 0.112   | 0.184        | 0.075  | 0.254   | 0.253     | 0.064           | 0.063           | 0.098             | <b>0.062</b> |
|           | sbs_02                  | 0.233    | 0.123   | 0.161        | 0.076  | 0.221   | 0.134     | 0.062           | <b>0.048</b>    | 0.116             | 0.061        |
|           | sbs_03                  | 0.281    | 0.122   | 0.142        | 0.070  | 0.309   | 0.132     | 0.061           | <b>0.047</b>    | 0.119             | 0.060        |
| Average   |                         | 7.416    | 0.097   | 0.151        | 0.278  | 1.928   | 0.137     | 0.051           | <b>0.044</b>    | 0.089             | 0.045        |

× denotes the system totally failed.

VO, often results in local optima or even negative optimization. Our LIO subsystem generally surpasses FAST-LIO2 due to our more accurate noise modeling for each LiDAR point. In a few sequences where FAST-LIO2 outperforms slightly, the differences are minimal, at the millimeter level, and negligible. Moreover, our system's accuracy significantly exceeds that of other tightly coupled LiDAR-inertial-visual systems across all sequences. Among them, LVI-SAM fails in nine sequences primarily due to its feature-based LIO and VIO subsystems not fully utilizing raw measurements, which degrades its robustness in environments with subtle geometric or texture features. R3LIVE generally performs well, but struggles in "Construction Stairs," "Cupola," and "Attic to Upper Gallery" sequences, where its performance is even worse than FAST-LIO2. This is because intense rotations at structure-less staircases result in inadequate pose priors, causing local optima when aligning colored map points with the current frame, and ultimately leading to negative optimization. FAST-LIVO and FAST-LIVO2 overcome such challenges in these sequences by the patch-based image alignments. In addition, situations where the sensors are close to walls in these sequences highlight the effectiveness of raycasting in FAST-LIVO2, with the mapping results in these large-scale scenes shown in Fig. S8 in the Supplementary Material [53]. On the other hand, FAST-LIVO is outperformed by R3LIVE and FAST-LIVO2 on the NTU-VIRAL dataset, especially in unstructured scenes, such as "nya" sequences, where the effects of affine warping based on constant depth assumptions are inaccurate. In contrast, the pixel-level alignment of R3LIVE and the plane prior (or refinement) of FAST-LIVO2 do not encounter such issues.

Comparing the different variants of FAST-LIVO2, we observe that the average accuracy without real-time exposure time estimation decreases by 6 mm compared to the default, as the

exposure time estimation can actively compensate illumination changes in the environment. On the other hand, the average accuracy without the reference patch update decreases by 44 mm compared to the default, as the reference patch update strategy effectively selects patches with higher resolution and avoids selecting outlier patches. Finally, the normal refinement increases the average accuracy by 1 mm, and the accuracy improvement is not consistent in all sequences. The limited improvement is mainly because normal vector refinement yields positive optimization only in simple structured scenes with nice image observations. In the NTU-VIRAL dataset, images from the "eee" and "nya" sequences are extremely dim and blurry, where negative optimization is particularly severe. To further study the effectiveness of the different modules, including exposure time estimation, affine warping, reference patch update, normal convergence, on-demand raycasting, and ESIKF sequential update, we conducted a thorough study on our private dataset and MARS-LVIG dataset. The results are presented in Section I (system module validation) in the Supplementary Material [53] due to the space limit. As confirmed in the results, our system can achieve robust and accurate pose estimation in both structured and unstructured environments, under severe light variations, in remarkably large-scale scenarios with long-term, high-speed data collection, and even in extremely narrow spaces with few LiDAR measurements.

### C. LiDAR Degenerated and Visually Challenging Environments

In this experiment, we evaluate the robustness of our system under environments experiencing LiDAR degeneration and/or visual challenges, comparing it with the qualitative mapping results of FAST-LIVO and R3LIVE in eight sequences, as shown



Fig. 10. Mapping results generated in real time in LiDAR degenerated scenes. The point clouds from top to bottom correspond to “Bright Screen Wall,” “Black Screen Wall,” “HIT Graffiti Wall” (the third and fourth rows), “Banner Wall,” “HKU Lecture Center,” respectively, showing the comparison of colored point cloud constructed FAST-LIVO2, FAST-LIVO, or R3LIVE (see more details on YouTube).<sup>10</sup>

in Figs. 10 and 11. Fig. 10 showcases LiDAR degeneration sequences where the LiDAR is facing a big wall while moving along the wall from one side to the other. Due to the absence of geometrical constraints since only one wall plane is being observed by the LiDAR, LIO methods would fail. It is worth mentioning that the “HIT Graffiti Wall” sequence spans nearly 800 m with LiDAR continuously facing the wall, leading to considerable degeneration. In all sequences, FAST-LIVO2 distinctly showcases its robustness against even long-term degeneration and its capability to deliver high-precision colored point maps. In contrast, FAST-LIVO managed to obtain the geometric structure but with completely blurred texture. R3LIVE struggles with both geometric structure and texture clarity. Fig. 11 showcases tests in more complicated scenarios where LiDAR and/or camera both degenerate occasionally. The degeneration directions are indicated by respective arrows. “HKU Cultural Center” [see Fig. 11(a)] showcases the mapping results of FAST-LIVO2, R3LIVE, and FAST-LIVO. As can be seen, R3LIVE and FAST-LIVO have distorted point maps, blurred textures, and drifts exceeding 1 m. In contrast, FAST-LIVO2 successfully returns to the starting point, achieving an impressive end-to-end error of less than 0.01 m, while achieving a consistent point map with clear textures. “CBD Building 03” [see Fig. 11(b)] and “Mining Tunnel” [see Fig. 11(c)] display only FAST-LIVO2 results, as R3LIVE and FAST-LIVO failed. In Fig. 11(b), the blue arrow represents movement toward a pure black screen, indicating concurrent LiDAR and camera degeneration. In Fig. 11(c1) and (c2), the red points represent the LiDAR scan at that location,

illustrating the areas of LiDAR degeneration due to the single plane being observed. Furthermore, the “Mining Tunnel” exhibits very dim lighting throughout the whole sequence, coupled with frequent visual and LiDAR degeneration. Despite of these challenges, FAST-LIVO2 still returns to the starting point with an end-to-end error of less than 0.01 m in both sequences.

#### D. High-Precision Mapping

In this experiment, we validate the high-precision mapping capabilities of our system. To explore the mapping accuracy across different algorithms and ensure fairness, we compare our system with FAST-LIO2, R3LIVE, and FAST-LIVO in scenes characterized by rich texture and structured environments. We take the sequences “SYSU 01,” “HKU Landmark,” and “CBD Building 01” as examples. Fig. S9 in the Supplementary Material [53] shows the colored point maps of these sequences reconstructed in real time. We can clearly observe that the point maps generated by FAST-LIVO2 retain the finest details among all the systems, the enlarged views of the colored point maps are akin to those in the actual RGB image. In the “SYSU 01” sequence, our algorithm produces fewer white noise dots on the signboard because we normalize the image colors to a reasonable exposure time using the recovered exposure time before coloring, resulting in rarely overexposed colored point maps. The reconstruction of the human and motorcycle in “CBD

<sup>10</sup><https://youtu.be/6dF2DzgbtIY>

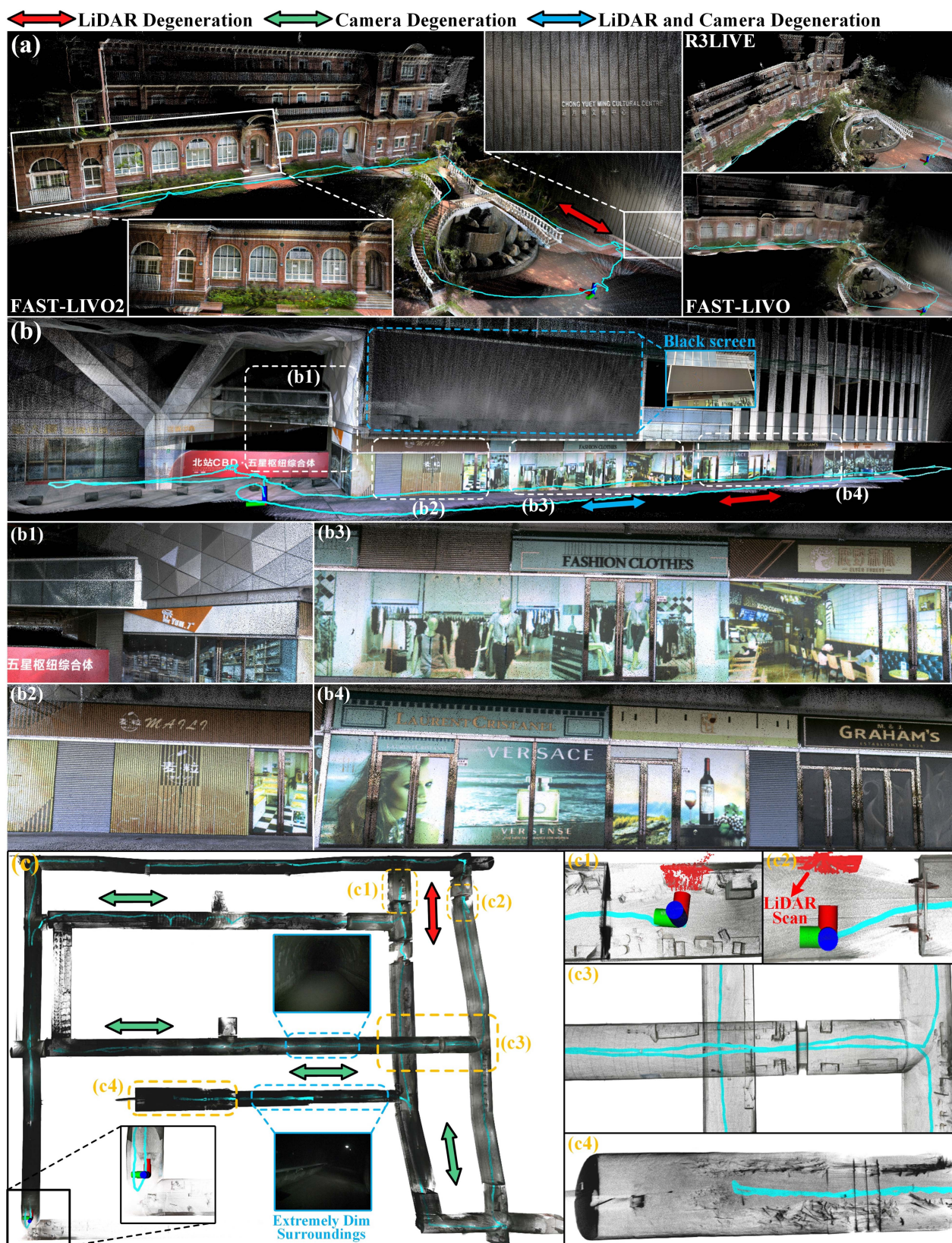


Fig. 11. Mapping results generated in real time in complex LiDAR degenerated and visually challenging scenes. (a), (b), and (c) correspond to “HKU Cultural Center,” “CBD Building 03,” and “Mining Tunnel,” respectively. Different colored arrows indicate the directions of degeneration caused by different sensors (see more details on YouTube).<sup>11</sup>

TABLE III  
 PROCESSING TIME (MS) PER LIDAR AND IMAGE FRAME

| Dataset                 | R3LIVE        | LVI-SAM       | FAST-LIVO    | FAST-LIVO2 (LiDAR / Image)   | FAST-LIVO2 (ARM) |
|-------------------------|---------------|---------------|--------------|------------------------------|------------------|
| <b>Hilti'22</b>         |               |               |              |                              |                  |
| Construction Ground     | 105.03        | ×             | 52.33        | <b>36.52 (20.44 / 15.05)</b> | 96.12            |
| Construction Multilevel | 112.13        | ×             | 56.12        | <b>38.77 (21.38 / 17.39)</b> | 95.38            |
| Construction Stairs     | 125.41        | 138.47        | 51.34        | <b>39.33 (24.32 / 15.01)</b> | 98.43            |
| Long Corridor           | 120.42        | 109.97        | 48.68        | <b>41.42 (26.21 / 15.21)</b> | 94.33            |
| Cupola                  | 151.52        | ×             | 59.42        | <b>43.54 (26.53 / 17.01)</b> | 98.12            |
| Lower Gallery           | 119.74        | 131.37        | 51.15        | <b>41.11 (25.69 / 15.42)</b> | 92.13            |
| Attic to Upper Gallery  | 144.39        | ×             | 58.61        | <b>44.21 (27.12 / 17.09)</b> | 91.15            |
| Outside Building        | 105.17        | 107.92        | 44.25        | <b>33.82 (18.91 / 14.91)</b> | 85.43            |
| <b>Hilti'23</b>         |               |               |              |                              |                  |
| Floor 0                 | 117.01        | ×             | 52.23        | <b>42.22 (25.13 / 17.09)</b> | 92.24            |
| Floor 1                 | 106.11        | 106.98        | 50.14        | <b>43.12 (27.43 / 15.69)</b> | 93.52            |
| Floor 2                 | 154.65        | ×             | 53.24        | <b>41.78 (26.12 / 15.66)</b> | 94.43            |
| Basement                | 118.21        | ×             | 48.23        | <b>39.65 (24.53 / 15.12)</b> | 93.42            |
| Stairs                  | 122.94        | 114.29        | 48.55        | <b>38.42 (22.64 / 15.78)</b> | 95.53            |
| Parking 3x floors down  | 142.43        | ×             | 51.89        | <b>43.62 (26.99 / 16.63)</b> | 94.22            |
| Large room              | 125.78        | 182.18        | 55.23        | <b>43.23 (26.43 / 16.80)</b> | 93.28            |
| Large room (dark)       | 131.31        | ×             | 51.46        | <b>44.21 (29.12 / 15.09)</b> | 92.29            |
| <b>NTU VIRAL</b>        |               |               |              |                              |                  |
| eee_01                  | 105.89        | 113.61        | 38.22        | <b>31.45 (17.24 / 14.21)</b> | 78.03            |
| eee_02                  | 112.27        | 119.19        | 39.43        | <b>30.24 (16.23 / 14.01)</b> | 79.22            |
| eee_03                  | 108.11        | 108.77        | 37.53        | <b>29.44 (16.12 / 13.32)</b> | 77.43            |
| nya_01                  | 122.73        | 124.54        | 39.66        | <b>34.52 (18.14 / 16.38)</b> | 79.44            |
| nya_02                  | 111.96        | 117.18        | 35.11        | <b>32.23 (17.33 / 14.90)</b> | 80.52            |
| nya_03                  | 115.96        | 111.24        | 38.42        | <b>33.42 (18.12 / 15.30)</b> | 78.95            |
| sbs_01                  | 112.77        | 115.67        | 39.23        | <b>31.92 (14.68 / 17.24)</b> | 78.32            |
| sbs_02                  | 113.01        | 110.37        | 35.39        | <b>32.56 (17.62 / 14.94)</b> | 72.43            |
| sbs_03                  | 119.91        | 120.73        | 37.41        | <b>33.62 (18.02 / 15.60)</b> | 75.56            |
| <b>Private Dataset</b>  |               |               |              |                              |                  |
| Retail Street           | 85.32         | 70.23         | 31.22        | <b>19.44 (10.42 / 9.02)</b>  | 64.12            |
| CBD Building 01         | 79.22         | 65.32         | 29.14        | <b>18.22 (10.32 / 7.90)</b>  | 62.33            |
| CBD Building 02         | ×             | ×             | ×            | <b>21.53 (11.50 / 10.03)</b> | 69.98            |
| CBD Building 03         | ×             | ×             | ×            | <b>21.89 (11.98 / 9.91)</b>  | 69.92            |
| HKU Landmark            | 75.43         | 68.77         | 28.33        | <b>17.42 (10.12 / 7.30)</b>  | 63.21            |
| HKU Lecture Center      | 70.42         | ×             | 31.45        | <b>18.12 (10.99 / 7.13)</b>  | 62.88            |
| HKU Centennial Garden   | 74.33         | 70.12         | 33.46        | <b>19.22 (10.80 / 8.42)</b>  | 64.52            |
| HKU Cultural Center     | 102.34        | ×             | 35.62        | <b>21.43 (11.42 / 10.01)</b> | 72.43            |
| HKU Main Building       | 75.62         | ×             | 30.21        | <b>19.43 (10.33 / 9.10)</b>  | 65.31            |
| HKUST Red Sculpture     | 84.22         | 90.13         | 32.43        | <b>20.14 (11.12 / 9.02)</b>  | 65.42            |
| HIT Graffiti Wall       | 112.56        | ×             | 39.98        | <b>21.71 (10.55 / 11.16)</b> | 68.99            |
| Banner Wall             | 74.13         | ×             | 30.22        | <b>19.32 (10.01 / 9.31)</b>  | 63.43            |
| Bright Screen Wall      | 73.23         | ×             | 28.43        | <b>18.22 (9.92 / 8.30)</b>   | 61.21            |
| Black Screen Wall       | 71.42         | ×             | 27.66        | <b>17.99 (9.10 / 8.89)</b>   | 60.91            |
| Office Building Wall    | ×             | ×             | ×            | <b>19.42 (10.11 / 9.31)</b>  | 62.33            |
| Narrow Corridor         | ×             | ×             | ×            | <b>18.44 (8.32 / 10.12)</b>  | 61.42            |
| Long Corridor           | ×             | ×             | ×            | <b>19.53 (10.43 / 9.10)</b>  | 62.55            |
| Mining Tunnel           | ×             | ×             | ×            | <b>29.43 (15.42 / 14.01)</b> | 79.32            |
| SYSU 01                 | 112.66        | 88.95         | 32.23        | <b>23.65 (12.51 / 11.14)</b> | 77.43            |
| SYSU 02                 | 110.12        | ×             | 32.12        | <b>22.63 (11.82 / 10.81)</b> | 72.31            |
| <b>Average</b>          | <b>108.36</b> | <b>108.45</b> | <b>41.43</b> | <b>30.03 (17.13 / 12.90)</b> | <b>78.44</b>     |

× denotes the system totally failed.

Building 01” also exemplifies our ability to rebuild the details of unstructured objects. In all sequences, the estimated final position returns to the starting point with an end-to-end error of less than 0.01 m. We also tested FAST-LIVO2 in the remaining sequences of the private dataset, with mapping results shown in Fig. S10–S13 in the Supplementary Material [53].

### E. Run Time Analysis

In this section, we evaluate the average computational time per LiDAR scan and image frame of our proposed system, tested on a desktop PC equipped with an Intel i7-10700K CPU and 32GB RAM. Our evaluations span public datasets including Hilti’22, Hilti’23, and NTU-VIRAL, and our private dataset. As shown in

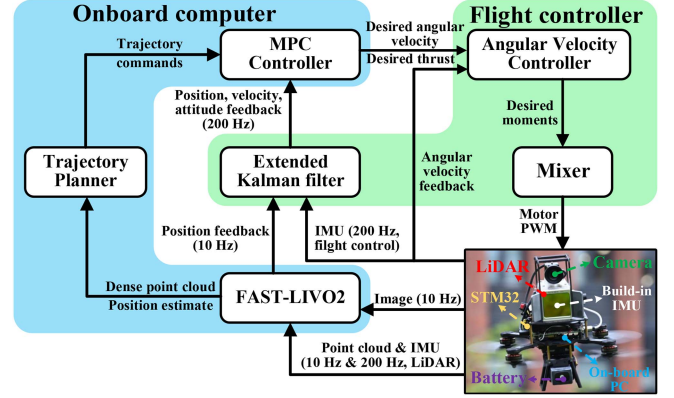


Fig. 12. Fully onboard UAV navigation algorithm flowchart.

Table III, our system exhibits the lowest processing time across all sequences. The average computation time consumption on an Intel i7 processor is only 30.03 ms (17.13 ms per LiDAR scan and 12.90 ms per image frame), fulfilling real-time operation at 10 Hz. Besides, our system can even operate in real time on ARM processors with an average processing time per frame of just 78.44 ms. LVI-SAM’s LiDAR and visual feature extraction modules in LIO and VIO are time-consuming. In addition to the time consumed by LIO and VIO, LVI-SAM integrates IMU preintegration constraints, visual odometry constraints, and LiDAR odometry constraints within a factor graph, further increasing the overall processing time. For R3LIVE, although also employing a direct method, its pixelwise image alignment necessitates the use of a large number of visual map points. In contrast, our approach uses sparse points with reference patches, enabling efficient alignment. In addition, R3LIVE maintains a colored map that undergoes Bayesian updating, significantly increasing the computational load as the map resolution increases. For FAST-LIVO2, the average processing time per frame (Table S3 in the Supplementary Material [53] due to space constraints) is approximately 10.35 ms less than FAST-LIVO2 due to not processing additional image measurements.

FAST-LIVO2 also shows noticeable improvements over the predecessor FAST-LIVO. The primary enhancement stems from our application of inverse compositional formulation in the sparse image alignment. Employing affine warping based on the plane prior from LiDAR points further enhances the convergence efficiency of our method. Consequently, FAST-LIVO2 reduces the number of iterations per pyramid level from 10 to 3, while still achieving superior accuracy.

## X. APPLICATIONS

To showcase the superior performance and versatility of FAST-LIVO2 in real-world applications, we develop multiple solutions, including fully onboard autonomous UAV navigation, airborne mapping, textured mesh generation, and 3-D Gaussian splatting (3DGS) reconstruction for 3-D scene representation.

### A. Fully Onboard Autonomous UAV Navigation

Given the high precision and robust localization performance of FAST-LIVO2, along with its real-time capabilities, we conduct closed-loop autonomous UAV flights.

<sup>11</sup><https://youtu.be/6dF2DzgbtIY>

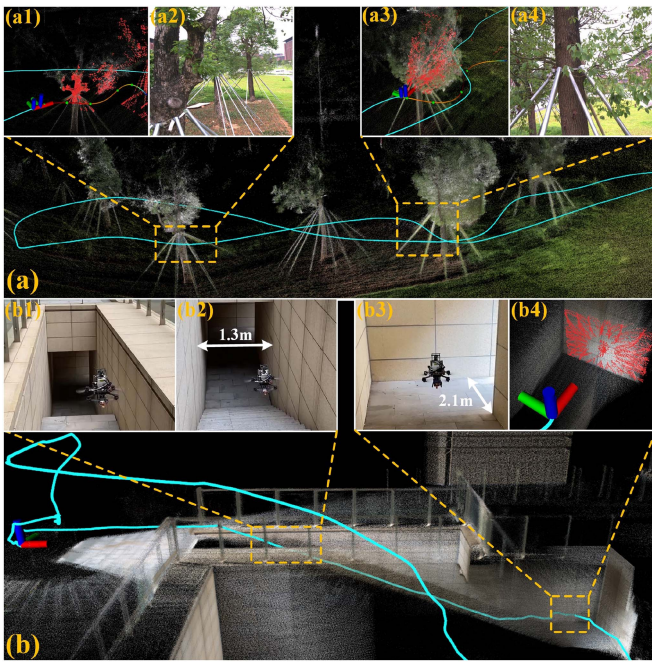


Fig. 13. (a) and (b) are the enlarged point maps of the “Woods” and “Narrow Opening” experiments, respectively. The red points in (a1), (a3), and (b4) represent the current scan. (a2) and (a4) represent the first-person view at the corresponding locations. (b1), (b2), and (b3) depict the third-person view (see more details on YouTube).<sup>12</sup>

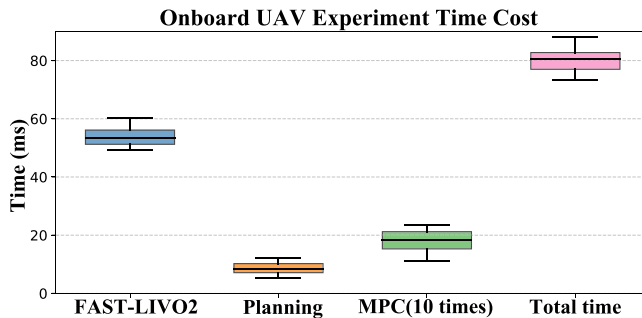


Fig. 14. Processing time of each module and in total in UAV autonomous navigation experiments across “Basement,” “Woods,” “Narrow Opening,” and “SYSU Campus.” The MPC executes at 100 Hz while planning and FAST-LIVO2 execute at 10 Hz, so its computation time is counted for ten times.

**1) System Configurations:** The hardware and software setup are illustrated in Fig. 12. For hardware, we use a NUC (Intel i7-1360P CPU and 32 GB RAM) as the onboard computer. In terms of software, the localization component is powered by FAST-LIVO2, which provides position feedback at 10 Hz. The localization result is fed to the flight controller to achieve 200 Hz feedback on position, velocity, and attitude. Besides localization, FAST-LIVO2 supplies a dense registered point cloud to the planning module, the Bubble planner [55], which plans a smooth trajectory that is then tracked by an on-manifold model predictive control (MPC) [56]. The MPC calculates the desired angular rates and thrust, which are tracked by respective low-level angular rate controllers running on the flight controller. Importantly, the MPC, Planner, and FAST-LIVO2 all operate on the onboard computer in real time.

<sup>12</sup><https://youtu.be/6dF2DzgblY>

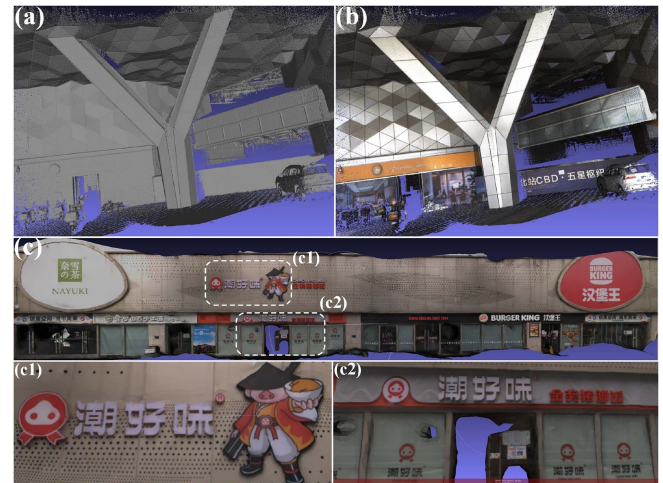


Fig. 15. (a) and (b) are the mesh and texture mapping of “CBD Building 01,” respectively. (c) is the texture mapping of “Retail Street,” with (c1) and (c2) showing local details.

**2) UAV Autonomous Navigation:** We conduct four fully onboard autonomous UAV navigation experiments, “Basement,” “Woods,” “Narrow Opening,” and “SYSU Campus” (Table S2 in the Supplementary Material [53]). “Basement” and “Woods” experiments are fully autonomous flights incorporating all planning, MPC, and FAST-LIVO2 modules, while “Narrow Opening” and “SYSU Campus” are manual flights with only MPC and FAST-LIVO2 (without the planning component). As can be seen, “Basement” and “Woods” showcase the UAV’s successful autonomous navigation and obstacle avoidance. In “Narrow Opening,” the UAV is commanded to fly close proximity to a wall leading to few LiDAR points measurements. Nevertheless, the raycasting module recalls a greater number of visual map points, providing abundant constraints for localization, which allows for stable localization. Moreover, “Basement” and “Narrow Opening” experience LiDAR degeneration, observing only a single wall [see Fig. 1(e1) and (e4), Fig. 13(b1)–(b4)], along with significant exposure variations [see Fig. 1(e5)–(e6)]. Despite these challenges, our UAV system performed exceptionally well. “Woods” involves the UAV moving at high speeds up to 3 m/s, demanding rapid response from the entire UAV system [see Fig. 13(a1)–(a4)]. “SYSU Campus,” a nondegenerated scene, primarily demonstrates the onboard high-precision mapping capabilities (see Fig. S14 in the Supplementary Material [53]). Finally, it is worth mentioning that in all these four UAV flights, severe lighting variation occurred. FAST-LIVO2 is able to estimate exposure time that closely follows the ground-truth values (see Fig. S15 in the Supplementary Material [53]).

Regarding onboard computational time, the need to run MPC (at 100 Hz) and Planning (at 10 Hz) on the onboard computer consumes computational resources and memory, limiting the computation resources available to FAST-LIVO2. Despite of the concurrent execution of control and planning, as illustrated in Fig. 14, the average onboard processing time per LiDAR scan and image frame for FAST-LIVO2, approximately 53.47 ms, is still well below the frame period 100 ms. The average processing times for planning and MPC are 8.43 and 18.5 ms, respectively. The total average processing time of 80.4 ms meets very well the real-time requirements for onboard operations.



Fig. 16. Comparison of ground-truth image, COLMAP+3DGS, and FAST-LIVO2+3DGS in terms of render details, computational time (time for generating point clouds and estimating poses + training time), and PSNR for a random frame in “CBD Building 01” (see more details on YouTube.<sup>13</sup>) (a) Ground-truth image. (b) COLMAP + 3DGS. (c) FAST-LIVO2 + 3DGS.

### B. Airborne Mapping

Airborne mapping represents a crucial task in surveying and mapping applications. To evaluate the suitability of FAST-LIVO2 for this application, we conduct an aerial mapping experiment using the public dataset MARS-LVIG [52] whose hardware configuration is detailed in Section VIII-A. We evaluate the two sequences “HKairport01” and “HKisland01,” whose real-time mapping results are illustrated in Fig. 1(a)–(c), with Fig. 1(a) and (c) corresponding to “HKisland01,” and (b) depicting “HKairport01.” The results demonstrate the effectiveness of FAST-LIVO2 in unstructured environments, such as forests and islands. The system successfully captures many fine structures and sharp coloring effects, including buildings, lane marks on roads, road curbs, tree crowns, and rocks, all of which are clearly visible. The APE (RMSE) for these sequences are 0.64 and 0.27 m for FAST-LIVO2, respectively, compared to 2.76 and 0.52 m for R3LIVE. The average processing times on the desktop PC (see Section IX-A), are approximately 25.2 and 21.8 ms, respectively, compared to 110.5 and 100.2 ms for R3LIVE.

### C. Supporting 3-D Scene Applications: Mesh Generation, Texture, and Gaussian Splatting

Leveraging the high-precision sensor localization and dense 3-D colored point map obtained from FAST-LIVO2, we develop software applications for rendering pipelines including meshing and texturing, as well as emerging NeRF-like rendering pipeline, such as 3DGS. For meshing, we employ VDBFusion [57] based on the truncated signed distance function in “CBD Building 01,” as shown in Fig. 15(a). The sharp edges on the columns and the distinct structure of the roof are clearly visible, demonstrating the high quality of the mesh. This level of detail is achieved due to the high density of FAST-LIVO2’s point clouds and the exceptional accuracy of structural reconstruction. After mesh construction, we use OpenMVS [58] to perform texture mapping using the estimated camera poses in “CBD Building 01” and “Retail Street,” as shown in Fig. 15(b)–(c). In Fig. 15(c1)–(c2), the texture images applied on the triangular facets are seamless and accurately aligned, resulting in a highly clear and precise texture mapping. This is attributed to a pixel-level image alignment achieved by FAST-LIVO2.

The dense color point clouds from FAST-LIVO2 can also directly serve as the input of 3DGS. We conduct tests on the sequence “CBD Building 01” utilizing 300 frames out of a total of 1180 images. The results are shown in Fig. 16. Compared to COLMAP [59], our method significantly reduces the time required to obtain dense point clouds and poses from 9 h to 21 s. However, the training time increases from 10 min 59 s to 15 min 30 s. This increase is attributed to the denser point clouds (downsampled to 5 cm), which introduce more parameters to optimize. Nonetheless, the increased density and precision of our point clouds result in a slightly higher peak signal-to-noise ratio (PSNR) compared to the PSNR obtained from COLMAP inputs.

## XI. CONCLUSION AND FUTURE WORK

This article proposed FAST-LIVO2, a direct LIVO framework achieving fast, accurate, and robust state estimation while reconstructing the map on the fly. FAST-LIVO2 can achieve high localization accuracy while being robust to severe LiDAR and/or visual degeneration.

The gain in speed is attributed to the use of raw LiDAR, inertial, and camera measurements within an efficient ESIKF framework with sequential update. In the image update, an inverse compositional formulation along with a sparse patch-based image alignment is further adopted to boost the efficiency. The gain in accuracy is attributed to the use (and even refine) of plane priors from LiDAR points to enhance accuracy of image alignment. Besides, a single unified voxel map is used to manage simultaneously the map points and the observed high-resolution image measurements. The voxel map structure, which supports geometry construction and update, visual map point generation and update, and reference patch update, is developed and validated. The gain in robustness is due to real-time estimation of exposure time, which effectively handles environment illumination variation, and on-demand voxel raycasting to cope with LiDARs’ close proximity blind zones. The efficiency and accuracy of FAST-LIVO2 were evaluated on extensive public datasets, while the robustness and effectiveness of each system module were evaluated on private dataset. The applications of FAST-LIVO2 in real-world robotics applications, such as

<sup>13</sup><https://youtu.be/6dF2DzgbtIY>

UAV navigation, 3-D mapping, and model rendering, were also demonstrated.

As an odometry, FAST-LIVO2 may have drifts over long distances. In the future, we could integrate loop closure and the sliding window optimization into FAST-LIVO2 to mitigate this long-term drift. Moreover, the accurate and dense colored point maps could be used to extract semantic information for object-level semantic mapping.

## REFERENCES

- [1] R. Mur-Artal and J. D. Tardós, "ORB-SLAM2: An open-source SLAM system for monocular, stereo, and RGB-D cameras," *IEEE Trans. Robot.*, vol. 33, no. 5, pp. 1255–1262, Oct. 2017.
- [2] J. Engel, V. Koltun, and D. Cremers, "Direct sparse odometry," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 40, no. 3, pp. 611–625, Mar. 2018.
- [3] J. Engel, T. Schöps, and D. Cremers, "LSD-SLAM: Large-scale direct monocular SLAM," in *Proc. Eur. Conf. Comput. Vis.*, 2014, pp. 834–849.
- [4] C. Forster, Z. Zhang, M. Gassner, M. Werlberger, and D. Scaramuzza, "SVO: Semidirect visual odometry for monocular and multicamera systems," *IEEE Trans. Robot.*, vol. 33, no. 2, pp. 249–265, Apr. 2017.
- [5] J. Zhang and S. Singh, "LOAM: Lidar odometry and mapping in real-time," in *Proc. Conf. Robot.: Sci. Syst.*, vol. 2, no. 9, 2014.
- [6] J. Lin and F. Zhang, "Loam livox: A fast, robust, high-precision lidar odometry and mapping package for lidars of small FOV," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2020, pp. 3126–3131.
- [7] T. Shan and B. Englot, "LeGO-LOAM: Lightweight and ground-optimized lidar odometry and mapping on variable terrain," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2018, pp. 4758–4765.
- [8] C. Zheng, Q. Zhu, W. Xu, X. Liu, Q. Guo, and F. Zhang, "FAST-LIVO: Fast and tightly-coupled sparse-direct lidar-inertial-visual odometry," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2022, pp. 4003–4009.
- [9] T. Qin, P. Li, and S. Shen, "VINS-Mono: A robust and versatile monocular visual-inertial state estimator," *IEEE Trans. Robot.*, vol. 34, no. 4, pp. 1004–1020, Aug. 2018.
- [10] R. Mur-Artal, J. M. M. Montiel, and J. D. Tardós, "ORB-SLAM: A versatile and accurate monocular SLAM system," *IEEE Trans. Robot.*, vol. 31, no. 5, pp. 1147–1163, Oct. 2015.
- [11] M. Irani and P. Anandan, "All about direct methods," in *Proc. Workshop Vis. Algorithms, Theory Pract.*, 1999, pp. 267–277.
- [12] C. Forster, M. Pizzoli, and D. Scaramuzza, "SVO: Fast semi-direct monocular visual odometry," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2014, pp. 15–22.
- [13] W. Xu, Y. Cai, D. He, J. Lin, and F. Zhang, "FAST-LIO2: Fast direct lidar-inertial odometry," *IEEE Trans. Robot.*, vol. 38, no. 4, pp. 2053–2073, Aug. 2022.
- [14] C. Yuan, W. Xu, X. Liu, X. Hong, and F. Zhang, "Efficient and probabilistic adaptive voxel mapping for accurate online lidar odometry," *IEEE Robot. Automat. Lett.*, vol. 7, no. 3, pp. 8518–8525, Jul. 2022.
- [15] M. Meilland, A. I. Comport, and P. Rives, "Real-time dense visual tracking under large lighting variations," in *Proc. Brit. Mach. Vis. Conf. Brit. Mach. Vis. Assoc.*, 2011, pp. 45–1.
- [16] T. Tykkälä, C. Audras, and A. I. Comport, "Direct iterative closest point for real-time visual odometry," in *Proc. IEEE Int. Conf. Comput. Vis. Workshops*, 2011, pp. 2050–2056.
- [17] C. Kerl, J. Sturm, and D. Cremers, "Robust odometry estimation for RGB-D cameras," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2013, pp. 3748–3754.
- [18] J. Engel, J. Sturm, and D. Cremers, "Semi-dense visual odometry for a monocular camera," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2013, pp. 1449–1456.
- [19] K. Chen, R. Nemiroff, and B. T. Lopez, "Direct lidar-inertial odometry: Lightweight LIO with continuous-time motion correction," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2023, pp. 3983–3989.
- [20] Z. Wang, L. Zhang, Y. Shen, and Y. Zhou, "D-LIOM: Tightly-coupled direct lidar-inertial odometry and mapping," *IEEE Trans. Multimedia*, vol. 25, pp. 3905–3920, 2023.
- [21] J. Zhang and S. Singh, "Laser-visual-inertial odometry and mapping with high robustness and low drift," *J. Field Robot.*, vol. 35, no. 8, pp. 1242–1264, 2018.
- [22] W. Shao, S. Vijayarangan, C. Li, and G. Kantor, "Stereo visual inertial lidar simultaneous localization and mapping," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2019, pp. 370–377.
- [23] J. Zhang, M. Kaess, and S. Singh, "A real-time method for depth enhanced visual odometry," *Auton. Robots*, vol. 41, pp. 31–43, 2017.
- [24] J. Graeter, A. Wilczynski, and M. Lauer, "LIMO: Lidar-monocular visual odometry," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2018, pp. 7872–7879.
- [25] Y. Zhu, C. Zheng, C. Yuan, X. Huang, and X. Hong, "CamVox: A low-cost and accurate lidar-assisted visual SLAM system," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2021, pp. 5049–5055.
- [26] S.-S. Huang, Z.-Y. Ma, T.-J. Mu, H. Fu, and S.-M. Hu, "Lidar-monocular visual odometry using point and line features," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2020, pp. 1091–1097.
- [27] C. Campos, R. Elvira, J. J. G. Rodríguez, J. M. Montiel, and J. D. Tardós, "ORB-SLAM3: An accurate open-source library for visual, visual-inertial, and multimap SLAM," *IEEE Trans. Robot.*, vol. 37, no. 6, pp. 1874–1890, Dec. 2021.
- [28] Y.-S. Shin, Y. S. Park, and A. Kim, "DVL-SLAM: Sparse depth enhanced direct visual-lidar SLAM," *Auton. Robots*, vol. 44, no. 2, pp. 115–130, 2020.
- [29] X. Zuo, P. Geneva, W. Lee, Y. Liu, and G. Huang, "LIC-Fusion: Lidar-inertial-camera odometry," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2019, pp. 5848–5854.
- [30] K. Sun et al., "Robust stereo visual inertial odometry for fast autonomous flight," *IEEE Robot. Automat. Lett.*, vol. 3, no. 2, pp. 965–972, Apr. 2018.
- [31] X. Zuo et al., "LIC-Fusion 2.0: Lidar-inertial-camera odometry with sliding-window plane-feature tracking," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2020, pp. 5112–5119.
- [32] D. Wisth, M. Camurri, S. Das, and M. Fallon, "Unified multimodal landmark tracking for tightly coupled lidar-visual-inertial odometry," *IEEE Robot. Automat. Lett.*, vol. 6, no. 2, pp. 1004–1011, Apr. 2021.
- [33] J. Lin, C. Zheng, W. Xu, and F. Zhang, "R2Live: A robust, real-time, lidar-inertial-visual tightly-coupled state estimator and mapping," *IEEE Robot. Automat. Lett.*, vol. 6, no. 4, pp. 7469–7476, Oct. 2021.
- [34] B. M. Bell and F. W. Cathey, "The iterated Kalman filter update as a Gauss-Newton method," *IEEE Trans. Autom. Control*, vol. 38, no. 2, pp. 294–297, Feb. 1993.
- [35] T. Shan, B. Englot, C. Ratti, and D. Rus, "LVI-SAM: Tightly-coupled lidar-visual-inertial odometry via smoothing and mapping," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2021, pp. 5692–5698.
- [36] J. Lin and F. Zhang, "R 3 live: A robust, real-time, RGB-colored, lidar-inertial-visual tightly-coupled state estimation and mapping package," in *Proc. Int. Conf. Robot. Automat.*, 2022, pp. 10672–10678.
- [37] J. Lin and F. Zhang, "R 3 Live++: A robust, real-time, radiance reconstruction package with a tightly-coupled lidar-inertial-visual state estimator," *IEEE Trans. Pattern Anal. Mach. Intell.*, 2024.
- [38] J. Engel, V. Usenko, and D. Cremers, "A photometrically calibrated benchmark for monocular visual odometry," 2016, *arXiv:1607.02555*.
- [39] W. Wang, J. Liu, C. Wang, B. Luo, and C. Zhang, "DV-Loam: Direct visual lidar odometry and mapping," *Remote Sens.*, vol. 13, no. 16, 2021, Art. no. 3340.
- [40] Z. Yuan, Q. Wang, K. Cheng, T. Hao, and X. Yang, "SDV-Loam: Semi-direct visual-lidar odometry and mapping," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 9, pp. 11203–11220, Sep. 2023.
- [41] H. Zhang, L. Du, S. Bao, J. Yuan, and S. Ma, "LVIO-Fusion: tightly-coupled lidar-visual-inertial odometry and mapping in degenerate environments," *IEEE Robot. Automat. Lett.*, vol. 9, no. 4, pp. 3783–3790, Apr. 2024.
- [42] W. Xu and F. Zhang, "FAST-LIO: A fast, robust lidar-inertial odometry package by tightly-coupled iterated Kalman filter," *IEEE Robot. Automat. Lett.*, vol. 6, no. 2, pp. 3317–3324, Apr. 2021.
- [43] D. He, W. Xu, and F. Zhang, "Symbolic representation and toolkit development of iterated error-state extended Kalman filters on manifolds," *IEEE Trans. Ind. Electron.*, vol. 70, no. 12, pp. 12533–12544, Dec. 2023.
- [44] D. Willner, C.-B. Chang, and K.-P. Dunn, "Kalman filter algorithms for a multi-sensor system," in *Proc. IEEE Conf. Decis. Control Including 15th Symp. Adaptive Processes*, 1976, pp. 570–574.
- [45] J. Ma and S. Sun, "Globally optimal distributed and sequential state fusion filters for multi-sensor systems with correlated noises," *Inf. Fusion*, vol. 99, 2023, Art. no. 101885.
- [46] Y. Ren, Y. Cai, F. Zhu, S. Liang, and F. Zhang, "ROG-MAP: An efficient robocentric occupancy grid map for large-scene and high-resolution lidar-based motion planning," 2023, *arXiv:2302.14819*.

- [47] R. M. Stereopsis, "Accurate, dense, and robust multiview stereopsis," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 32, no. 8, pp. 1362–1376, Aug. 2010.
- [48] S. Baker and I. Matthews, "Lucas-Kanade 20 years on: A unifying framework," *Int. J. Comput. Vis.*, vol. 56, pp. 221–255, 2004.
- [49] T.-M. Nguyen, S. Yuan, M. Cao, Y. Lyu, T. H. Nguyen, and L. Xie, "NTU VIRAL: A visual-inertial-ranging-lidar dataset, from an aerial vehicle viewpoint," *Int. J. Robot. Res.*, vol. 41, no. 3, pp. 270–280, 2022.
- [50] M. Helmberger, K. Morin, B. Berner, N. Kumar, G. Cioffi, and D. Scaramuzza, "The Hilti SLAM challenge dataset," *IEEE Robot. Automat. Lett.*, vol. 7, no. 3, pp. 7518–7525, Jul. 2022.
- [51] L. Zhang et al., "Hilti-Oxford dataset: A millimeter-accurate benchmark for simultaneous localization and mapping," *IEEE Robot. Automat. Lett.*, vol. 8, no. 1, pp. 408–415, Jan. 2023.
- [52] H. Li et al., "MARS-LVIG dataset: A multi-sensor aerial robots SLAM dataset for lidar-visual-inertial-GNSS fusion," *Int. J. Robot. Res.*, vol. 43, no. 8, 2024, Art. no. 02783649241227968.
- [53] "Supplementary material: Fast-livo2: Fast, direct lidar-inertial-visual odometry," Aug. 2024. [Online]. Available: [https://github.com/hku-mars/FAST-LIVO2/blob/main/Supplementary/LIVO2\\_supplementary.pdf](https://github.com/hku-mars/FAST-LIVO2/blob/main/Supplementary/LIVO2_supplementary.pdf)
- [54] C. Klug, C. Arth, D. Schmalstieg, and T. Gloor, "Measurement uncertainty analysis of a robotic total station simulation," in *Proc. IECON 44th Annu. Conf. IEEE Ind. Electron. Soc.*, 2018, pp. 2576–2582.
- [55] Y. Ren et al., "Bubble planner: Planning high-speed smooth quadrotor trajectories using receding corridors," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2022, pp. 6332–6339.
- [56] G. Lu, W. Xu, and F. Zhang, "On-manifold model predictive control for trajectory tracking on robotic systems," *IEEE Trans. Ind. Electron.*, vol. 70, no. 9, pp. 9192–9202, Sep. 2023.
- [57] I. Vizzo, T. Guadagnino, J. Behley, and C. Stachniss, "VDBFusion: Flexible and efficient TSDF integration of range sensor data," *Sensors*, vol. 22, no. 3, 2022, Art. no. 1296. [Online]. Available: <https://www.mdpi.com/1424-8220/22/3/1296>
- [58] D. Cernea, "OpenMVS: Multi-view stereo reconstruction library 2020," 2020. [Online]. Available: <https://cdsacave.github.io/openMVS>
- [59] J. L. Schönberger and J.-M. Frahm, "Structure-from-motion revisited," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 4104–4113.



**Zuhao Zou** received the B.Eng. degree in automatic control system engineering from The University of Sheffield, Sheffield, U.K., in 2016, and the M.Sc. degree in computer vision, graphic, and imaging from University College London, London, U.K., in 2017. He is currently working toward the Ph.D. degree in mechanical engineering with Hong Kong University, Hong Kong, China.

His research interests include robotics, sensor fusion, localization and mapping, and loop detection.



**Tong Hua** received the B.S. degree in electronic engineering from Shanghai Jiao Tong University, Shanghai, China, in 2021. He has been working toward the master's degree in information and communication engineering with the Shanghai Key Laboratory of Navigation and Location Based Services, Shanghai Jiao Tong University, Shanghai, China, since 2021.

His research interests include multisensor fusion and visual inertial odometry.



**Chongjian Yuan** received the B.Eng. degree in automation from the College of Control Science and Engineering, Zhejiang University, Hangzhou, China, in 2016. He is currently working toward the Ph.D. degree in robotics with the Department of Mechanical Engineering, The University of Hong Kong, Hong Kong, China.

His research interests include light detection and ranging SLAM and sensor fusion.



**Dongjiao He** (Member, IEEE) received the B.S. degree from Southeast University, Nanjing, China, in 2016, the M.S. degree from Shanghai Jiao Tong University, Shanghai, China, in 2019, both in mechanical engineering, and the Ph.D. degree in robotics from the University of Hong Kong, Hong Kong, China, in July 2024, under supervision of Prof. Fu Zhang.

Her research interests of the doctoral degree include advanced multisensor algorithms for autonomous navigation, SLAM, and other related fields of UAVs. She is committed to advancing research for

autonomous systems, including multisource information fusion, autonomous perception, and strategy decision.



**Bingyang Zhou** (Member, IEEE) received the B.Eng. degree in automation from the Harbin Institute of Technology, Shenzhen, China, in 2023. He is currently working toward the M.Phil. degree in mechanical engineering with MaRS Lab, Department of Mechanical Engineering, The University of Hong Kong, Hong Kong, China.

His research focuses on simultaneous localization and mapping.



**Chunran Zheng** (Student Member, IEEE) received the B.Eng. degree in automation from Xian Jiaotong University, Xi'an, China, in 2020. He is currently working toward the Ph.D. degree in robotics and SLAM with the Department of Mechanical Engineering, The University of Hong Kong, Hong Kong, China.

His research interests include multisensor calibration, sensor fusion, LiDAR-inertial-visual SLAM, and 3-D Gaussian splatting.



**Wei Xu** received the Ph.D. degree in mechanical engineering from the University of Hong Kong, Hong Kong, China, in 2022.

He is currently the Chief Technology Officer with Manifold Tech Limited, Hong Kong, China. His research and expertise include filtering algorithms, multisensor fusion, simultaneous localization and dense mapping, unmanned aerial vehicle systems, and motion planning.



**Zheng Liu** (Member, IEEE) received the B.Eng. degree in automation from the Harbin Institute of Technology, Harbin, China, in 2019, and the Ph.D. degree in mechanical engineering from the University of Hong Kong, Hong Kong, China, in 2024.

His research interests include visual or LiDAR-based localization and mapping, and sensor fusion and calibration.



**Rong Wang** received the B.Sc. degree in automation from Beihang University, Beijing, China, 2013, and the Ph.D. degree in automation from the University of Chinese Academy of Sciences, Beijing, China, in June 2018.

Her Ph.D. work focused on SLAM and augmented reality. From July 2018, she was with the Information Science Academy, China Electronics Technology Group Corporation, Beijing, China. Her research interests include semantic SLAM and UAV.



**Jiarong Lin** (Member, IEEE) received the B.S. degree in optical information science and technology from the University of Electronic Science and Technology of China, Chengdu, China, in 2015. He is currently working toward the Ph.D. degree in robotics with the Department of Mechanical Engineering, The University of Hong Kong, Hong Kong, China.

His research interests include light detection and ranging mapping and sensor fusion.



**Fanle Meng** received the B.Sc. degree in biomedical engineering from the Huazhong University of Science and Technology, Wuhan, China, in 2010, and the Ph.D. degree in biomedical engineering from Tsinghua University, Beijing, China, in June 2018.

Her Ph.D. work focused on medical robotics and surgery navigation. From July 2018, she was with the Information Science Academy, China Electronics Technology Group Corporation, Beijing, China. Her research interests include SLAM and UGV.



**Fangcheng Zhu** (Student Member, IEEE) received the B.Eng. degree in automation from the School of Mechanical Engineering and Automation, Harbin Institute of Technology, Shenzhen, China, in 2021. He is currently working toward the Ph.D. degree in robotics with the Department of Mechanical Engineering, University of Hong Kong, Hong Kong, China.

His research interests include LiDAR-based simultaneous localization and mapping, sensor calibration, and aerial swarm systems.



**Fu Zhang** (Member, IEEE) received the B.E. degree in automation from the University of Science and Technology of China, Hefei, China, in 2011, and the Ph.D. degree in controls from the University of California, Berkeley, CA, USA, in 2015.

His Ph.D. work focused on self-calibration and control of microrate-integrating gyrosensors. In 2016, he was a Research Assistant Professor in design and control of unmanned aerial vehicles (UAVs) with the Robotics Institute, Hong Kong University of Science and Technology, Hong Kong, China. From Aug 2018, he was an Assistant Professor with the Department of Mechanical Engineering, University of Hong Kong, Hong Kong, China. His research interests include robotics and controls, with focus on UAV design, navigation, control, and lidar-based simultaneous localization and mapping.



**Yunfan Ren** (Student Member, IEEE) received the B.Eng. degree in automation from the Harbin Institute of Technology, Harbin, China, in 2021. He is currently working toward the Ph.D. degree with the Department of Mechanical Engineering, The University of Hong Kong, Hong Kong, China.

He is a Member of the MaRS Lab, Hong Kong, China. His research interests include autonomous navigation, UAV planning, optimal control, and swarm system autonomy.

Mr. Ren was recognized as an Outstanding Navigation Paper Finalist at ICRA 2023 and as both the Best Overall and Best Student Paper Finalist at IROS 2023.